Unit 1

Natural Language Processing (NLP):

It is a subset of artificial intelligence, computer science, and linguistics focused on making human communication, such as speech and text, comprehensible to computers.

NLP is used in a wide variety of everyday products and services. Some of the most common ways NLP is used are through voice-activated digital assistants on smartphones, email-scanning programs used to identify spam, and translation apps that decipher foreign languages.

Natural Language Techniques:

NLP encompasses a wide range of techniques to analyse human language. Some of the most common techniques you will likely encounter in the field include:

- **Sentiment analysis:** An NLP technique that analyses text to identify its sentiments, such as "positive," "negative," or "neutral." Sentiment analysis is commonly used by businesses to better understand customer feedback.

- **Summarization:** An NLP technique that summarizes a longer text, in order to make it more manageable for time-sensitive readers. Some common texts that are summarized include reports and articles.

- **Keyword extraction:** An NLP technique that analyses a text to identify the most important keywords or phrases. Keyword extraction is commonly used for search engine optimization (SEO), social media monitoring, and business intelligence purposes.

- **Tokenization:** The process of breaking characters, words, or sub-words down into "tokens" that can be analysed by a program. Tokenization undergirds common NLP tasks like word modelling, vocabulary building, and frequent word occurrence.

NLP Benefits:

- The ability to analyse both structured and unstructured data, such as speech, text messages, and social media posts.
- Improving customer satisfaction and experience by identifying insights using sentiment analysis.
- Reducing costs by employing NLP-enabled AI to perform specific tasks, such as chatting with customers via chatbots or analysing large amounts of text data.
- Better understanding a target market or brand by conducting NLP analysis on relevant data like social media posts, focus group surveys, and reviews.

NLP Limitations:

NLP can be used for a wide variety of applications but it's far from perfect. In fact, many NLP tools struggle to interpret sarcasm, emotion, slang, context, errors, and other types of ambiguous statements.

This means that NLP is mostly limited to unambiguous situations that don't require a significant amount of interpretation.

NLP Examples:

Although natural language processing might sound like something out of a science fiction novel, the truth is that people already interact with countless NLP-powered devices and services every day.

Online chatbots, for example, use NLP to engage with consumers and direct them toward appropriate resources or products. While chat bots can’t answer every question that customers may have, businesses like them because they offer cost-effective ways to troubleshoot common problems or questions that consumers have about their products.

Another common use of NLP is for text prediction and autocorrect, which you’ve likely encountered many times before while messaging a friend or drafting a document. This technology allows texters and writers alike to speed-up their writing process and correct common typos.

NLP Applications:

Natural Language Processing (NLP) has a wide range of applications across various industries and domains. Here are some real-world examples of how NLP is being used:

1. Sentiment Analysis : NLP is used to analyse social media comments, customer reviews, and feedback to determine public sentiment towards a product, brand, or topic. Companies can use this information to make informed decisions about their products or services.

2. Chatbots and Virtual Assistants : Virtual assistants like Siri, Alexa, and Google Assistant rely heavily on NLP to understand and respond to user voice commands and natural language queries. Chatbots are also used in customer support to answer common questions and handle basic tasks.

3. Language Translation : Services like Google Translate use NLP to translate text or speech from one language to another. This technology is essential for breaking down language barriers in a globalized world.

4. Text Summarization : NLP is used to automatically generate summaries of long texts, which is particularly useful in news aggregation, research, and content curation.

5. Speech Recognition : NLP is used in speech recognition systems, enabling applications like voice assistants, transcription services, and voice command systems in automobiles.

Some Common NLP Tasks:

Chunking	Syntactic Parsing	Machine Translation
Part of Speech Tagging	Word Sense Disambiguation	Text Generation
Named Entity Recognition	Sentiment Analysis	Automatic Summarization
Speech Detection	Topic Modeling	Question Answering
Thesaurus	Information Retrieval	Conversational Interfaces (ChatBot)

Linguistic Fundamentals:

Linguistic fundamentals play a foundational role in the field of Natural Language Processing (NLP), which is dedicated to enabling computers to understand, process, and generate human language. These fundamentals are essential for building effective NLP systems and ensuring accurate and context-aware language understanding.

Components of Natural language:

- **Phonetics:** Phonetics is about the acoustic and articulatory properties of the sounds which can be produced by the human vocal tract, particularly those which are utilized in the sound systems of languages.
- **Phonology:** Phonology concerns the use of sounds in a particular language. English makes use of about 45 phonemes – contrastive sounds
- **Morphology:** Morphology concerns the structure and meaning of words. Some words, such as send, appear to be 'atomic' or monomorphemic. Others, such as sends, sending, resend appear to be constructed from several atoms or morphemes
- **Lexicon**: refer to the component of an NLP system that contains information (semantic, grammatical) about individual words or word strings. **EX:** what sound or orthography goes with what meaning. That is, what part-of-speech a word is, **EX:** storm can be noun or verb.
- **Syntax**: Syntax concerns the way in which words can be combined together to form (grammatical) sentences;
- **Semantics**: Semantics is about the manner in which lexical meaning is combined morphologically and syntactically to form the meaning of a sentence. Mostly, this is regular, productive, and rule-governed.

Grammar and Inference:

- Linguists tend to use the term grammar in an **extended sense to cover all the structure of human languages:** phonology, morphology, syntax, and their contribution to meaning.
- However, even if you know the grammar of a language, in this sense, you still need more knowledge to interpret many utterances.
- All of the following, sentences are underspecified in this sense. Pronouns, ellipsis (incomplete sentences) and other ambiguities of various kinds all **requires additional non-grammatical information to select an appropriate interpretation** given the (extra)linguistic context.

Regular Expression (RE):

A regular expression (RE) is a language for specifying text search strings. RE helps us to match or find other strings or sets of strings, using a specialized syntax held in a pattern.

Regular Expressions	**Regular Set**
(0 + 10*)	{0, 1, 10, 100, 1000, 10000, … }
(0*10*)	{1, 01, 10, 010, 0010, …}
(0 + ε)(1 + ε)	{ε, 0, 1, 01}
(a+b)*	It would be set of strings of a's and b's of any length which also includes the null string i.e. {ε, a, b, aa , ab , bb , ba, aaa…….}
(a+b)*abb	It would be set of strings of a's and b's ending with the string abb i.e. {abb, aabb, babb, aaabb, ababb, ……..}
(11)*	It would be set consisting of even number of 1's which also includes an empty string i.e. {ε, 11, 1111, 111111, …….}
(aa)*(bb)*b	It would be set of strings consisting of even number of a's followed by odd number of b's i.e. {b, aab, aabbb, aabbbbb, aaaab, aaaabbb, ……..}

Words & Corpora:

- A corpus is a large and structured set of machine-readable texts that have been produced in a natural communicative setting. (Its plural is corpora.)
- It can be derived in different ways like text that was originally electronic, transcripts of spoken language and optical character recognition, etc.
- **Corpus Balance:** A balanced corpus covers a wide range of text categories, which are supposed to be representatives of the language. We do not have any reliable scientific measure for balance but the best estimation and intuition works in this concern. In other words, we can say that the accepted balance is determined by its intended uses only.
- **Corpus Sampling:** According to **Biber(1993)**, "Some of the first considerations in constructing a corpus concern the overall design: for example, the kinds of texts included, the number of texts, the selection of particular texts, the selection of text samples from within texts, and the length of text samples. Each of these involves a sampling decision, either conscious or not."
- While obtaining a representative sample, we need to consider the following −
    - **Sampling unit** − It refers to the unit which requires a sample. For example, for written text, a sampling unit may be a newspaper, journal, or a book.
    - **Sampling frame** − The list of all sampling units is called a sampling frame.
    - **Population** − It may be referred as the assembly of all sampling units. It is defined in terms of language production, language reception or language as a product.
- **Corpus Size:** Another important element of corpus design is its size. How large the corpus should be? There is no specific answer to this question. The size of the corpus depends upon the purpose for which it is intended as well as on some practical considerations as follows:
    - Kind of query anticipated from the user.
    - The methodology used by the users to study the data.
    - Availability of the source of data.

Tokenization and Segmentation in NLP:

1. Tokenization:

Tokenization is the process of dividing a text into individual units called tokens. Tokens are typically words, sub-words, or symbols, and they serve as the basic building blocks for NLP tasks. Tokenization helps in extracting meaningful information from text and is a necessary step for various applications in NLP. Here are some key points about tokenization:

a. **Word Tokenization:** In most NLP applications, words are the primary tokens. Word tokenization splits text into words based on spaces or punctuation. For example, the sentence "Tokenization is important!" would be tokenized into ["Tokenization", "is", "important", "!"].
b. **Sub-word Tokenization:** Sub-word tokenization splits text into smaller units, such as sub-word pieces or characters. This approach is useful for languages with complex word formations, agglutinative languages, or when dealing with out-of-vocabulary words. Techniques like Byte-Pair Encoding (BPE) and Sentence Piece are examples of sub-word tokenization.
c. **Token Normalization:** Tokenization also involves normalizing tokens by converting them to lowercase or applying other transformations to ensure consistency in text processing.
d. **Handling Special Cases:** Tokenization should account for special cases, like contractions ("I'm" should be tokenized as ["I", "'m"]), hyphenated words ("mother-in-law" should be treated as a single token), and abbreviations ("Dr." should be a single token).
e. **Punctuation Handling:** Tokens should include punctuation marks as separate tokens when necessary. For instance, "Mr. Smith" should be tokenized into ["Mr.", "Smith"].

2. Segmentation:

Segmentation is the process of dividing text into meaningful units, often used in languages with little to no explicit word boundaries. Asian languages, such as Chinese, Japanese, and Korean, rely heavily on segmentation because their writing systems don't use spaces between words. Here are some key points about segmentation:

a. **Word Segmentation:** In languages like Chinese, where words are written as continuous sequences of characters, word segmentation is crucial. It involves identifying word boundaries within a sentence. For example, the Chinese sentence "我喜欢NLP" would be segmented into ["我", "喜欢", "NLP"], where each segment represents a word.
b. **Sentence Segmentation:** Sentence segmentation is the process of splitting a text into individual sentences. This is vital for tasks like machine translation, sentiment analysis, and text summarization. Sentence boundaries are often marked by punctuation marks like periods, exclamation marks, or question marks.
c. **Segmentation Challenges:** In languages like Japanese and Thai, segmentation can be challenging because there are no spaces or clear sentence boundaries. Tokenization in such languages may involve using language-specific dictionaries and statistical models to determine word and sentence boundaries.

Text Normalization:

Why do we need text normalization?

- When we normalize text, we attempt to reduce its randomness, bringing it closer to a predefined "standard".
- This helps us to reduce the amount of different information that the computer has to deal with.
- It also improves efficiency of NLP application.
- The goal of normalization techniques like **stemming** and **lemmatization** is to reduce inflectional forms.
- Also derivationally related forms of a word to a common base form.

Techniques for Text Normalization?

A. Contractions:

- **Contractions** are words or combinations of words that are shortened by dropping letters and replacing them by an apostrophe, and removing them contributes to text standardization.

B. Removing Punctuation:

Punctuation marks, such as periods, commas, question marks, and exclamation points, serve as important elements of written language but are typically not essential for many NLP tasks. Removing punctuation is a common preprocessing step for the following reasons:

- **Noise Reduction:** Punctuation marks can introduce noise in the text data, making it more challenging for NLP algorithms to extract meaningful information.
- **Consistency:** Removing punctuation ensures that text is consistent in its representation, which can be important for tasks like text classification and sentiment analysis.
- **Tokenization:** Tokenization, which is the process of splitting text into words or tokens, is simplified when punctuation is removed because words are separated more cleanly.

However, it's important to note that some punctuation marks, such as hyphens, apostrophes in contractions, and certain symbols in technical documents, should be handled with care and not removed indiscriminately.

C. Handling Capitalization:

Capitalization refers to the use of uppercase and lowercase letters in text. Managing capitalization is essential in NLP for the following reasons:

- **Normalization:** Converting all text to either lowercase or uppercase can help in normalizing the data, making it consistent for analysis. Lowercasing is a common choice as it treats "word" and "Word" as the same, reducing the vocabulary size.
- **Case-Sensitive Tasks:** In some NLP tasks, capitalization carries meaning. For instance, Named Entity Recognition (NER) relies on capitalization to identify proper nouns, like "New York City."
- **Preserving Information:** In situations where capitalization is important (e.g., distinguishing between "apple" and "Apple"), you should carefully consider whether to retain or normalize capitalization.

D. Stop Words Handling:

Stop words are common words such as "the," "a," "an," "in," "to," and "and" that occur frequently in text but often carry little meaningful information. Managing stop words is essential for several reasons:

- **Noise Reduction:** Removing stop words helps reduce noise in the text data and focuses the analysis on more meaningful content.
- **Computational Efficiency:** Stop words consume memory and computational resources but don't contribute significantly to NLP tasks like text classification, sentiment analysis, or information retrieval. Removing them can improve efficiency.
- **Customization:** In some cases, depending on the specific NLP task, you may choose to retain certain stop words that are contextually relevant.

E. Stemming:

What is Stemming?

Stemming is an elementary rule-based process for removing inflectional forms from a token and the outputs are the **stem** of the world.

For example, “laughing”, “laughed”, “laughs”, “laugh” will all become “laugh”, which is their stem, because their inflection form will be removed.

“laughing”, “laughed”, “laughs”, “laugh” >>> “laugh”

Stemming is not a good normalization process because sometimes stemming can produce words that are not in the dictionary. For example, consider a sentence: “His teams are not winning”

After stemming the tokens that we will get are- “hi”, “team”, “are”, “not”, “winn”

Notice that the keyword **“winn”** is not a regular word and “**hi**” changed the context of the entire sentence.

Another example could be-

Form	Suffix	Stem
studies	-es	studi
studying	-ing	study

Applications of Stemming:

- Stemming is used in information retrieval systems like search engines.
- It is used to determine domain vocabularies in domain analysis.
- To display search results by indexing while documents are evolving into numbers and to map documents to common subjects by stemming.

F. Lemmatization:

What is Lemmatization?

Lemmatization, on the other hand, is a systematic step-by-step process for removing inflection forms of a word. It makes use of vocabulary, word structure, part of speech tags, and grammar relations.

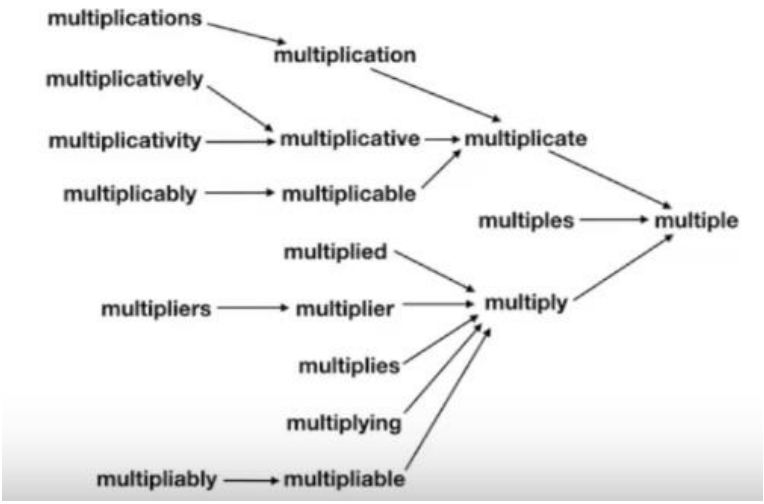
The output of lemmatization is the root word called **a lemma**. For example,

Am, Are, Is >> Be

Running, Ran, Run >> Run

Also, since it is a systematic process while performing lemmatization one can specify the part of the speech tag for the desired term and lemmatization will only be performed if the given word has the proper part of the speech tag. For example, if we try to lemmatize the word **running** as a **verb**, it will be converted to **run.** But if we try to lemmatize the same word **running** as a **noun** it won't be converted.

A detailed explanation of how Lemmatization works by the step-by-step process to remove inflection forms of a word-

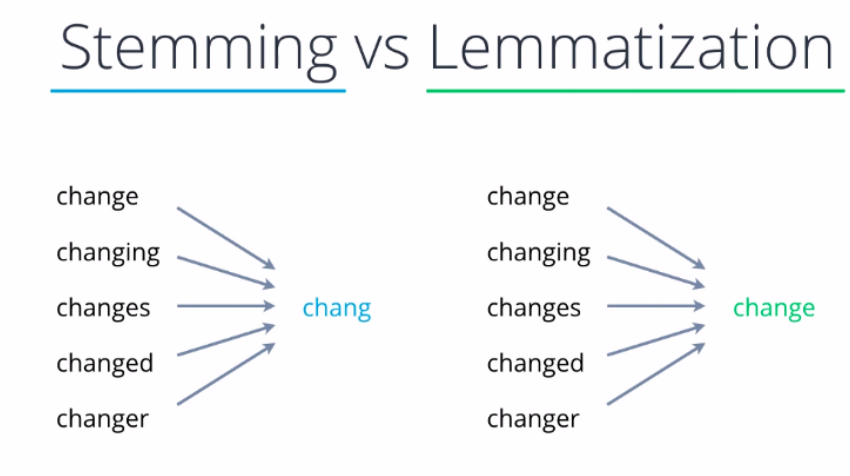


Applications of Lemmatization:

- **Biomedicine:** Using lemmatization to parse biomedicine literature may increase the efficiency of data retrieval tasks.
- **Search engines**
- **Compact indexing:** Lemmatization is an efficient method for storing data in the form of index values.

Stemming vs Lemmatization:

Stemming	Lemmatization
Stemming is a process that stems or removes last few characters from a word, often leading to incorrect meanings and spelling.	**Lemmatization** considers the context and converts the word to its meaningful base form, which is called Lemma.
For instance, stemming the word 'Caring' would return 'Car'.	For instance, lemmatizing the word '**Caring**' would return '**Care**'.
Stemming is used in case of large dataset where performance is an issue.	Lemmatization is computationally expensive since it involves look-up tables and what not.



G. Part of Speech(POS) Tags in NLP:

Part of speech tags or POS tags is the properties of words that define their main context, their function, and the usage in a sentence. Some of the commonly used parts of speech tags are- **Nouns**, which define any object or entity; **Verbs**, which define some action; and **Adjectives** or **Adverbs**, which act as the modifiers, quantifiers, or intensifiers in any sentence. In a sentence, every word will be associated with a proper part of the speech tag, for example,

"David has purchased a new laptop from the Apple store."

In the below sentence, every word is associated with a part of the speech tag which defines their functions.

In this case "David' has **NNP** tag which means it is a proper noun, "has" and "purchased" belongs to verb indicating that they are the actions and "laptop" and "Apple store" are the nouns, "new" is the adjective whose role is to modify the context of laptop.

Part of speech tags is defined by the relations of words with the other words in the sentence. Machine learning models or rule-based models are applied to obtain the part of speech tags of a word. The most commonly used part of speech tagging notations is provided by the Penn Part of Speech Tagging.

Part of speech tags have a large number of applications and they are used in a variety of tasks such as **text cleaning, feature engineering tasks, and word sense disambiguation**. For example, consider these two sentences-

Sentence 1: "Please **book** my flight for NewYork"

Sentence 2: "I like to read a **book** on NewYork"

In both sentences, the keyword "book" is used but in sentence one, it is used as a verb while in sentence two it is used as a noun.

Minimum Edit Distance:

- Many NLP tasks are concerned with measuring ***how similar two strings are***.
- Spell correction:
  - The user typed "graffe"
  - Which is closest? : **graf grail giraffe**
    - the word **giraffe**, which differs by only one letter from **graffe**, seems intuitively to be more similar than, say **grail** or **graf**,
- The **minimum edit distance** between two strings is defined as the ***minimum numberof editing operations*** (**insertion**, **deletion**, **substitution**) needed to transform one string into another.
- The **minimum edit distance** between **intention**and **execution** can be visualized using their alignment.
- Given two sequences, an **alignment** is a correspondence between substrings of the twosequences.

```
I N T E * N T I O N
| | | | | | | | | |
* E X E C U T I O N
d s s   i s
```

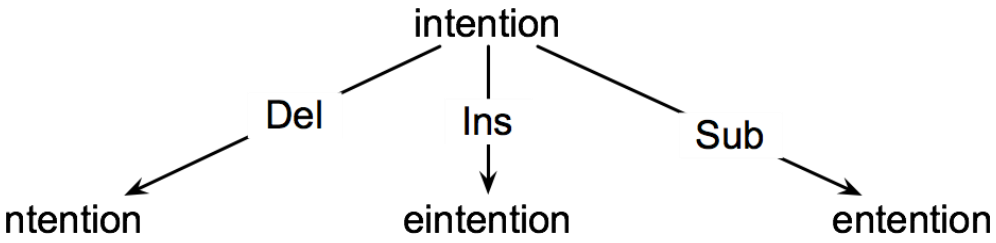
- If each operation has cost of 1
  - Distance between them is 5
- If substitutions cost 2 (**Levenshtein Distance**)
  - Distance between them is 8
- Evaluating Machine Translation and speech recognition

R Spokesman confirms senior government adviser was shot

H Spokesman said the senior adviser was shot dead

S I D I

- How do we find the minimum edit distance?
  - We can think of this as a **search task**, in which we are searching for **the shortest path**—a sequence of edits—from one string to another.



- The space of all possible edits is enormous, so we can't search naively.
  - Most of distinct edit paths ends up in the same state, so rather than recomputing all thosepaths, we could just remember ***the shortest path to a state*** each time we saw it.
  - We can do this by using **dynamic programming**.
  - **Dynamic programming** is the name for a class of algorithms that apply a table-driven method to solve problems by combining solutions to sub-problems.

- For two strings
  - the source string X of length **n**
  - the target string Y of length **m**

- We define **D(i,j)** as the **edit distance** between X[1..i] and Y[1..j]
    - i.e., the first **i** characters of X and the first **j** characters of Y

- The **edit distance between X and Y** is thus **D(n,m)**

Computing Minimum Edit Distance:

- We will compute D(n,m) **bottom up**, combining solutions to subproblems.

- Compute **base cases** first:
  - D(i,0) = i
    - a source substring of length i and an empty target string requires i deletes.
  - D(0,j) = j
    - a target substring of length j and an empty source string requires j inserts.

- Having computed D(i,j) for small i, j we then compute larger D(i,j) based onpreviously computed smaller values.

- The value of D(i, j) is computed by taking the minimum of the three possible paths through the matrix which arrive there:

$$D[i,j] = \min \begin{cases} D[i-1,j] + \text{del-cost}(source[i]) \\ D[i,j-1] + \text{ins-cost}(target[j]) \\ D[i-1,j-1] + \text{sub-cost}(source[i], target[j]) \end{cases}$$

- If we assume the version of **Levenshtein distance** in which the insertions and deletions each have a cost of 1, and substitutions have a cost of 2 (except substitutionof identical letters have zero cost), the computation for D(i,j) becomes:

$$D[i,j] = \min \begin{cases} D[i-1,j] + 1 \\ D[i,j-1] + 1 \\ D[i-1,j-1] + \begin{cases} 2; & \text{if } source[i] \neq target[j] \\ 0; & \text{if } source[i] = target[j] \end{cases} \end{cases}$$

Minimum Edit Distance Algorithm:

```
function MIN-EDIT-DISTANCE(source, target) returns min-distance

  n ← LENGTH(source)
  m ← LENGTH(target)
  Create a distance matrix distance[n+1,m+1]

  # Initialization: the zeroth row and column is the distance from the empty string
      D[0,0] = 0
      for each row i from 1 to n do
          D[i,0] ← D[i-1,0] + del-cost(source[i])
      for each column j from 1 to m do
          D[0,j] ← D[0,j-1] + ins-cost(target[j])

  # Recurrence relation:
  for each row i from 1 to n do
      for each column j from 1 to m do
          D[i,j] ← MIN( D[i−1,j] + del-cost(source[i]),
                        D[i−1,j−1] + sub-cost(source[i],target[j]),
                        D[i,j−1] + ins-cost(target[j]))
  # Termination
  return D[n,m]
```

Computation of Minimum Edit Distance between intention and execution:

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 & \text{deletion} \\ D(i,j-1) + 1 & \text{insertion} \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} & \text{substitution} \end{cases}$$

N	9	8	9	10	11	12	11	10	9	8
O	8	7	8	9	10	11	10	9	8	9
I	7	6	7	8	9	10	9	8	9	10
T	6	5	6	7	8	9	8	9	10	11
N	5	4	5	6	7	8	9	10	11	10
E	4	3	4	5	6	7	8	9	10	9
T	3	4	5	6	7	8	7	8	9	8
N	2	3	4	5	6	7	8	7	8	7
I	1	2	3	4	5	6	7	6	7	8
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

- Edit distance isn't sufficient
  - We often need to align each character of the two strings to each other
- We do this by keeping a "**backtrace**"
- Every time we enter a cell, remember where we came from
- When we reach the end,
  - Trace back the path from the upper right corner to read off the alignment

MinEdit with Backtrace:

N	9									
O	8									
I	7		$D(i,j) = \min \begin{cases} D(i-1,j) + 1 & \text{deletion} \\ D(i,j-1) + 1 & \text{insertion} \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} & \text{substitution} \end{cases}$							
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

n	9	↓ 8	↙←↓ 9	↙←↓ 10	↙←↓ 11	↙←↓ 12	↓ 11	↓ 10	↓ 9	**↙ 8**
o	8	↓ 7	↙←↓ 8	↙←↓ 9	↙←↓ 10	↙←↓ 11	↓ 10	↓ 9	**↙ 8**	← 9
i	7	↓ 6	↙←↓ 7	↙←↓ 8	↙←↓ 9	↙←↓ 10	↓ 9	**↙ 8**	← 9	← 10
t	6	↓ 5	↙←↓ 6	↙←↓ 7	↙←↓ 8	↙←↓ 9	**↙ 8**	← 9	← 10	←↓ 11
n	5	↓ 4	↙←↓ 5	↙←↓ 6	↙←↓ 7	**↙←↓ 8**	↙←↓ 9	↙←↓ 10	↙←↓ 11	↙↓ 10
e	4	↙ 3	← 4	**↙← 5**	**← 6**	← 7	←↓ 8	↙←↓ 9	↙←↓ 10	↓ 9
t	3	↙←↓ 4	**↙←↓ 5**	↙←↓ 6	↙←↓ 7	↙←↓ 8	↙ 7	←↓ 8	↙←↓ 9	↓ 8
n	2	**↙←↓ 3**	↙←↓ 4	↙←↓ 5	↙←↓ 6	↙←↓ 7	↙←↓ 8	↓ 7	↙←↓ 8	↙ 7
i	**1**	↙←↓ 2	↙←↓ 3	↙←↓ 4	↙←↓ 5	↙←↓ 6	↙←↓ 7	↙ 6	← 7	← 8
#	**0**	1	2	3	4	5	6	7	8	9
	#	**e**	**x**	**e**	**c**	**u**	**t**	**i**	**o**	**n**

Adding Backtrace to Minimum Edit Distance:

- Base conditions: Termination:

D(i,0) = i D(0,j) = j D(N,M) is distance

- Recurrence Relation:

```
For each  i = 1…M
     For each  j = 1…N
                          D(i-1,j) + 1          deletion
               D(i,j)= min D(i,j-1) + 1          insertion
                          D(i-1,j-1) +  2; if X(i) ≠ Y(j)   substitution
                                        0; if X(i) = Y(j)

                          LEFT          insertion
               ptr(i,j)=  DOWN          deletion
                          DIAG          substitution
```

Weighted Minimum Edit Distance:

In the context of NLP, WMED extends the idea of MED by assigning different weights or costs to different edit operations. This means that not all edit operations are considered equally costly. Instead, each operation is associated with a weight that reflects its importance or relevance in a particular linguistic or domain-specific context.

Here are some key points to understand about Weighted Minimum Edit Distance in NLP:

1. Operation Weights: In NLP applications, certain edit operations may be more or less significant than others. For example, in spelling correction, a substitution of a single character might be less costly than deleting a whole word or inserting a new word. WMED allows us to assign weights to these operations to better reflect their importance.

2. Cost Functions: To calculate WMED, cost functions are used to determine the cost associated with each type of edit operation. These cost functions can be predefined based on linguistic or domain-specific knowledge. For instance, a substitution of a vowel with another vowel might have a lower cost than substituting a consonant with a vowel.

3. Dynamic Programming: WMED is often computed using dynamic programming algorithms, similar to the algorithms used for MED. However, in WMED, the dynamic programming matrix is extended to include the operation weights. This dynamic programming matrix helps find the optimal sequence of edit operations with the minimum total cost.

4. Applications: WMED finds applications in various NLP tasks, such as machine translation, spell checking, text summarization, and information retrieval. It helps in determining the most likely sequence of edits that transforms one sequence of words into another, considering the specific costs associated with different operations.

5. Example: Suppose you have two sentences, "I have an apple" and "I have a pineapple," and you want to find the WMED between them. You might assign a lower weight to the operation of substituting 'apple' with 'a pineapple' compared to deleting 'apple' and inserting 'a pineapple.' WMED takes these weights into account when computing the distance.

Unit 2

N-gram Models:

Language modelling is the way of determining the probability of any sequence of words. Language modelling is used in a wide variety of applications such as Speech Recognition, Spam filtering, etc. In fact, language modelling is the key aim behind the implementation of many state-of-the-art Natural Language Processing models.

Methods of Language Modelling:
Two types of Language Modelling:

- **Statistical Language Modelling**: Statistical Language Modelling, or Language Modelling, is the development of probabilistic models that are able to predict the next word in the sequence given the words that precede. Examples such as N-gram language modelling.
- **Neural Language Modelling**: Neural network methods are achieving better results than classical methods both on standalone language models and when models are incorporated into larger models on challenging tasks like speech recognition and machine translation. A way of performing a neural language model is through word embeddings.

Statistical language modelling	Neural language modelling
• Probabilistic models. • Predicts the next word in a sequence. • Can be used for disambiguating the input. • Used for selecting a probable solution. • Depends on the theory of probability.	• Gives better results than the classical methods both for the standalone models and when the models are incorporated into the larger models on the challenging tasks. • E. g. Speech recognitions and machine translations. • Word embedding.

What is an N-gram model?

The N-gram model is a statistical language model that estimates the probability of the next word in a sequence based on the previous N-1 words. It works on the assumption that the probability of a word depends only on the preceding N-1 words, which is known as the Markov assumption. In simpler terms, it predicts the likelihood of a word occurring based on its context.

N-grams are essentially sequences of N words or characters. For example, a bigram consists of two words, while a trigram consists of three words. These sequences are then used to calculate the probability of a particular word given its preceding context. In a bigram model, the probability of a word is based on its preceding word only, while in a trigram model, the probability is based on the two preceding words.

How does the N-gram Language Model work?

The N-gram language model works by calculating the frequency of each N-gram in a large corpus of text. The frequency of these N-grams is then used to estimate the probability of a particular word given its context.

For example, let's consider the sentence, "I am going to the grocery store". We can generate bigrams from this sentence by taking pairs of adjacent words, such as "I am", "am going", "going to", "to the", and "the grocery". The frequency of each bigram in a large corpus of text can be calculated, and these frequencies can be used to estimate the probability of the next word given the previous word.

A bigram model is a type of n-gram model where n=2. This means that the model calculates the probability of each word in a sentence based on the previous word.

To calculate the probability of the next word given the previous word(s) in a bigram model, we use the following formula:

$$P(w_n \mid w_{n-1}) = \text{count}(w_{n-1}, w_n) / \text{count}(w_{n-1})$$

where $P(w_n \mid w_{n-1})$ is the probability of the nth word given the previous word, and $\text{count}(w_{n-1}, w_n)$ is the count of the bigram (w_{n-1}, w_n) in the text, and $\text{count}(w_{n-1})$ is the count of the previous word in the text.

For example, let's say we have the sentence: "The cat sat on the mat." In a bigram model, we would calculate the probability of each word based on the previous word:

P("The" | Start) = 1 (assuming "Start" is a special token indicating the start of a sentence)
P("cat" | "The") = 1
P("sat" | "cat") = 1
P("on" | "sat") = 1
P("the" | "on") = 1
P("mat" | "the") = 0.5 (assuming "the" appears twice in the sentence)

The probability of each word given the previous word can be used to generate new sentences or to evaluate the likelihood of a given sentence.

A trigram model is similar to a bigram model, but it calculates the probability of each word based on the previous two words. To calculate the probability of the next word given the previous two words in a trigram model, we use the following formula:

$$P\left(w_i | w_{i-(n-1)}, \ldots, w_{i-1}\right) = \frac{count\left(w_{i-(n-1)}, \ldots, w_{i-1}, w_i\right)}{count\left(w_{i-(n-1)}, \ldots, w_{i-1}\right)}$$

or

P(w_n | w_{n-2}, w_{n-1}) = count(w_{n-2}, w_{n-1}, w_n) / count(w_{n-2}, w_{n-1})

where P(w_n | w_{n-2}, w_{n-1}) is the probability of the nth word given the previous two words, and count(w_{n-2}, w_{n-1}, w_n) is the count of the trigram (w_{n-2}, w_{n-1}, w_n) in the text, and count(w_{n-2}, w_{n-1}) is the count of the previous two words in the text.

For example, let's say we have the sentence: "I love to eat pizza." In a trigram model, we would calculate the probability of each word based on the previous two words:

P("I" | Start, Start) = 1
P("love" | Start, "I") = 0
P("to" | "I", "love") = 1
P("eat" | "love", "to") = 1
P("pizza" | "to", "eat") = 1

Again, the probability of each word given the previous two words can be used to generate new sentences or to evaluate the likelihood of a given sentence. However, trigram models may be less accurate than bigram models because they require more context to calculate the probability of each word.

Limitations of N-gram Model in NLP:

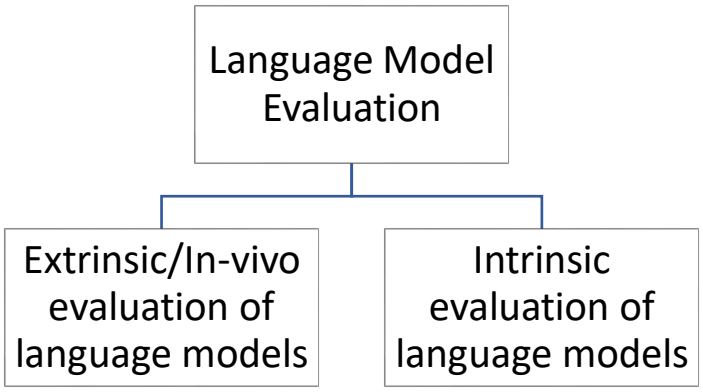
The N-gram language model has also some limitations:

- There is a problem with the out of vocabulary words. These words are during the testing but not in the training. One solution is to use the fixed vocabulary and then convert out vocabulary words in the training to pseudowords.
- When implemented in the sentiment analysis, the bi-gram model outperformed the uni-gram model but the number of the features is then doubled. So, the scaling of the N-gram model to the larger data sets or moving to the higher-order needs better feature selection approaches.
- The N-gram model captures the long-distance context poorly. It has been shown after every 6-grams, the gain of performance is limited.

Language Model Evaluation:

How good is your language model?

To answer the above questions for language models, we first need to answer the following intermediary question: Does our language model assign a higher probability to grammatically correct and frequent sentences than those sentences which are rarely encountered or have some grammatical error? To train parameters of any model we need a training dataset. After training the model, we need to evaluate how well the model's parameters have been trained; for which we use a test dataset which is utterly distinct from the training dataset and hence unseen by the model. After that, we define an evaluation metric to quantify how well our model performed on the test dataset.



1. Extrinsic/In-vivo evaluation of language models:

For comparing two language models A and B, pass both the language models through a specific natural language processing task and run the job. After that compare the accuracies of models A and B to evaluate the models in comparison to one another. The natural language processing task may be text summarization, sentiment analysis and so on.

Limitations: Time consuming mode of evaluation.

2. **Intrinsic evaluation of language models:**
   - **Perplexity:**

     Perplexity is the multiplicative inverse of the probability assigned to the test set by the language model, normalized by the number of words in the test set. If a language model can predict unseen words from the test set, i.e., the P(a sentence from a test set) is highest; then such a language model is more accurate.

Chain rule:

$$\mathrm{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i|w_1 \ldots w_{i-1})}}$$

For bigrams:

$$\mathrm{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i|w_{i-1})}}$$

As a result, better language models will have lower perplexity values or higher probability values for a test set.

Smoothing Techniques:

What is Smoothing in NLP?

In NLP, we have statistical models to perform tasks like auto-completion of sentences, where we use a probabilistic model. Now, we predict the next words based on training data, which has complete sentences so that the model can understand the pattern for prediction. Naturally, we have so many combinations of words possible. It is next to impossible to include all the varieties in training data so that our model can predict accurately on unseen data. So, here comes Smoothing to the rescue.

Smoothing refers to the technique we use to adjust the probabilities used in the model so that our model can perform more accurately and even handle the words absent in the training set.

Why do we need smoothing in NLP?

We use Smoothing for the following reasons.

- To improve the accuracy of our model.
- To handle data sparsity, out of vocabulary words, words that are absent in the training set.

Example - Training set: ["I like coding", “Prakriti likes mathematics”, “She likes coding”]

Let’s consider bigrams, a group of two words.

$P(w_i \mid w_{(i-1)}) = count(w_i\ w_{(i-1)}) / count(w_{(i-1)})$

So, let's find the probability of “I like mathematics”.

We insert a start token, <S> and end token, <E> at the start and end of a sentence respectively.

P(“I like mathematics”)

= P(I | <S>) * P(like | I) * P(mathematics | like) * P(<E> | mathematics)

= (count(<S>I) / count(<S>)) * (count(I like) / count(I)) * (count(like mathematics) / count(like)) * (count(mathematics <E>) / count(<E>))

= (1/3) * (1/1) * (0/1) * (1/3)

=0

As you can see, P (“I like mathematics”) comes out to be 0, but it can be a proper sentence, but due to limited training data, our model didn’t do well.
Now, we’ll see how smoothing can solve this issue.

Types of Smoothing in NLP:

1. **Laplace / Add-1 Smoothing:**

   Here, we simply add 1 to all the counts of words so that we never incur 0 value.

   $P_{Laplace}(w_i \mid w_{(i-1)}) = (count(w_i\ w_{(i-1)}) + 1) / (count(w_{(i-1)}) + V)$

   Where V= total words in the training set, 9 in our example.

   So, P("I like mathematics")

   = P(I | <S>)*P(like | I)*P(mathematics | like)*P(<E> | mathematics)

   = ((1+1) / (3+9)) * ((1+1) / (1+9)) * ((0+1) / (1+9)) * ((1+1) / (3+9))

   = 1 / 1800

2. **Add K Smoothing:**

   $P_{add\text{-}k}(w_i \mid w_{(i-1)}) = count(w_i\ w_{(i-1)}) + k / count(w_{(i-1)}) + k.V)$

   Choosing right value of k is a tedious job. So not prefer a lot.

3. **Backoff:**

   It says using less content is good.

   - Start with n-gram,
     - If insufficient observations, check (n-1)gram
     - If insufficient observations, check (n-2)gram

4. **Interpolation:**

   Try a mixture of (multiple) n-gram models

   $$P(w_i \mid w_{i-2}w_{i-1}) = \alpha_1 P(w_i \mid w_{i-2}w_{i-1}) + \alpha_2 P(w_i \mid w_{i-1}) + \alpha_3 P(w_i), \quad \text{subjected to } \sum \alpha_i = 1$$

5. **Good Turing Smoothing:**

   This technique uses the frequency of occurring of N-grams reallocates probability distribution using two criteria.

   For example, as we saw above, P("like mathematics") equals 0 without smoothing. We use the frequency of bigrams that occurred once, the total number of bigrams for unknown bigrams.

   $P_{unknown}(w_i \mid w_{(i-1)})$ = (count of bigrams that appeared once) / (count of total bigrams)

   For known bigrams like "like coding," we use the frequency of bigrams that occurred more than one of the current bigram frequency (N_{c+1}), frequency of bigrams that occurred the same as the current bigram frequency (N_c), and the total number of bigrams(N).

   $P_{known}(w_i \mid w_{(i-1)}) = c^* / N$

   Where $c^* = (c+1) * (N_{c+1}) / (N_c)$ and c = count of input bigram, "like coding" in our example.

Vector Semantics:

In NLP, we would like to have a model that explains various aspects of words such as word similarity, word senses, lexical semantics, and so on.

1. **Word Similarity:**
   Word similarity refers to the degree of resemblance or likeness between two or more words in terms of their meaning. It quantifies how closely related or interchangeable words are in a given context. Word similarity can be measured using various techniques, including computational methods like cosine similarity or word embeddings (e.g., Word2Vec or GloVe) that capture semantic relationships between words based on their co-occurrence patterns in large text corpora. High word similarity suggests that words share similar meanings or are semantically related, while low similarity implies less semantic overlap.

2. **Word Senses:**
   In natural language, many words have multiple meanings or senses. Word senses refer to the different interpretations or definitions that a word can have, depending on the context in which it is used. These different senses can be quite distinct or subtly related. For example, the word "bank" can refer to a financial institution (e.g., a bank where you deposit money) or the side of a river (e.g., the bank of a river). Identifying and disambiguating word senses is crucial for tasks like natural language understanding, machine translation, and information retrieval.

3. **Lexical Semantics:**
   Lexical semantics is a subfield of linguistics and computational linguistics that focuses on the study of word meaning, particularly how words convey meaning in context. It explores how words relate to each other, their sense distinctions, and the semantic relationships that exist between words in a language. Lexical semantics delves into questions such as synonymy (the relationship between synonyms), antonymy (the relationship between antonyms), hypernymy (the relationship between a more general word and its more specific instances), and hyponymy (the relationship between a specific instance and the general category it belongs to). Lexical semantics also examines phenomena like polysemy (a word having multiple related senses) and homonymy (a word having multiple unrelated senses). Computational techniques and resources, such as lexical databases and semantic similarity measures, are used to model and study lexical semantics in computational linguistics.

How can we build a computation model that explains meaning of a word? The answer is Vector semantics. Vector Semantics defines semantics & interprets word meaning to explain features such as word similarity. Its central idea is: Two words are similar if they have similar word context.

Why Vector Model?

In its current form, the vector model inspires its working from linguistic and philosophical work of 1950s. Vector semantics represents a word in multi-dimensional vector space. Vector model is also called Embeddings, due to the fact that word is embedded in a particular vector space. Vector model offers many advantages in NLP. For example, in sentimental analysis, it sets up a boundary class and predicts if the sentiment is positive or negative (a binomial classification). Another key practical advantage with vector semantics is that it can learn automatically from text without complex labelling or supervision. As a result of these advantages, the vector semantics has become a de-facto standard for NLP applications such as Sentiment Analysis, Named Entity Recognition (NER), topic modelling, and so on.

Type of Word Embedding:

1. Term Document (TD) Matrix:

A vector or distributional model of word meaning is based on co-occurrence matrix. One way of building co-occurrence matrix is using Term Document (TD) frequency model. Term Document (TD) represents the frequency of a given work in a document. For example, the following is a TD matrix of 4 sample words from 4 Shakespeare plays(1):

Plays →	As You Like It	Twelfth Night	Julius Caesar	Henry V
Battle	1	0	7	13
Good	114	80	62	89
Fool	36	58	1	4
Wit	20	15	2	3

Table 1

TD vector was originally devised for Information Retrieval (IR), wherein you are required to find out similar documents for a given query. For example, in the above table, you can see that plays "As you Like It" and "Twelfth Night" might be similar based on column vector entries — comparing As you Like It [1,114,36,20] with Twelfth Night [0, 80, 58, 15].

Of course, here, we have sampled only 4 words. That is, the vocabulary size is 4 and the vector will have a dimension of |V| = 4. In a typical NLP application, wherein you consider thousands of documents, you will see this matrix will be a long and sparse — mostly with entries 0s.

Another way of constructing co-occurrence matrix is using Term Context (TC) matrix. In this case, word similarity is obtained by comparing word with other words in a given document or in a batch of documents. In TC matrix, typically, you would consider a moving window of comparison among words. For example, consider the following 2 short documents:

doc = ["I like machine learning", "I love NLP"]

In this case, the vocabulary size is |V| = 6 and the dimension of the matrix is |V| x |V| — in this case, 6 x 6. For this toy example, the TC matrix with a window size of 1 is:

	I	like	machine	learning	love	NLP
I	0	1	0	0	1	0
like	1	0	1	0	0	1
machine	0	1	0	1	0	0
learning	0	0	1	0	0	0
love	1	0	0	0	0	1
NLP	0	0	0	0	1	0

Table 2

Of course, a typical NLP application will have a vocabulary size ranging in 1000s. So, in TC matrix too, the matrix is too large and sparse.

Both these co-occurrence matrices show the frequency of an item associated with documents. It turns out that word frequency alone wouldn't be enough to understand its importance– whether a word is discriminative or not. Traditionally, we believe that more frequent words are more important than infrequent words. However, there are frequent words such as "good" in table 1 above are unimportant. How do you balance these two contrast expectations — capture all frequent words yet they should be discriminative? TF-IDF answer this paradox question.

2. Term Frequency, Inverse Document Frequency (TF-IDF):

The tf-idf is a product of two terms: term frequency (tf) and inverse document frequency (idf). The TF defines the frequency of a given term in a given document. In an NLP application, since the frequency of a term might be too high, we down weight the frequency term by applying log10 scale. So, TF is defined as:

$$tf_{(t,d)} = \begin{cases} 1 + log_{10} count(t,d) & for\ count(t,d) > 0 \\ 0 & Else \end{cases}$$

Inverse Document Frequency(idf) assigns higher weights to words that occur only in few documents. Such words are quite useful for discriminating those documents from the rest of the collection. The idf is defined as N/dft, where N is number of documents and dft is document frequency — the number of documents in which the term (t) occurs. Here too, we apply the log10 scale to down weight the IDF value:

$$idf_t = log_{10}(N/df_t)$$

We can now compute the tf-idf weight vector:

$$w_{t,d} = tf_{t,d} \times idf_t$$

As you can see here, the tf-idf appropriately measures the importance of a word and helps us identify whether a given word is discriminative or not.

As stated at the beginning, the biggest disadvantage with co-occurrence model is that it results in a long, sparse matrix. NLP applications typically prefer a dense matrix with lower sparsity. In spite of its sparse nature, the tf-idf vector model is still useful in applications such as Information retrieval. Traditionally, NLP specialists use TF-IDF model for baseline metric before trying out more advanced topics.

Pros and Cons of TF-IDF Vectorization:

Though TF-IDF is an improvement over the simple bag of words approach and yields better results for common NLP tasks, the overall pros and cons remain the same. We still need to create a huge sparse matrix, which also takes a lot more computation than the simple bag of words approach.

3. Word2Vec:

Word2Vec approach uses deep learning and neural networks-based techniques to convert words into corresponding vectors in such a way that the semantically similar vectors are close to each other in N-dimensional space, where N refers to the dimensions of the vector.

Word2Vec returns some astonishing results.

The mathematical details of how Word2Vec works involve an explanation of neural networks and SoftMax probability

Word2Vec model comes in two flavors: Skip Gram Model and Continuous Bag of Words Model (CBOW).

Pros and Cons of Word2Vec:

Word2Vec has several advantages over bag of words and IF-IDF scheme. Word2Vec retains the semantic meaning of different words in a document. The context information is not lost.

Another great advantage of Word2Vec approach is that the size of the embedding vector is very small. Each dimension in the embedding vector contains information about one aspect of the word. We do not need huge sparse vectors, unlike the bag of words and TF-IDF approaches.

Pointwise Mutual Information (PMI):

Use cases of NLP can be seen across industries like understanding customers' issues, predicting the next word user is planning to type in the keyboard, automatic text summarization etc. Many researchers across the world trained NLP models in several human languages like English, Spanish, French, Mandarin etc. so that benefit of NLP can be seen in every society. It is the most useful NLP metric called Pointwise mutual information (PMI) to identify words that can go together along.

What is Pointwise mutual information?

PMI helps us to find related words. In other words, it explains how likely the co-occurrence of two words than we would expect by chance. For example, the word "Data Science" has a specific meaning when these two words "Data" and "Science" go together. Otherwise meaning of these two words are independent. Similarly, "Great Britain" is meaningful since we know the word "Great" can be used with several other words but not so relevant in meaning like "Great UK, Great London, Great Dubai etc."

When words 'w1' and 'w2' are independent, their joint probability is equal to the product of their individual probabilities. Imagine when the formula of PMI as shown below returns 0, it means the numerator and denominator is same and then taking log of 1 produces 0. In simple words it means the words together has NO specific meaning or relevance. Question arises what are we trying to achieve here. We are focusing on the words which have high joint probability with the other word but having not so high probability of occurrence if words are considered separately. It implies that this word pair has a specific meaning.

$$\mathrm{PMI}(w_1, w_2) = \log_2 \frac{P(w_1, w_2)}{P(w_1)P(w_2)}$$

$$P(w) = \frac{Freq(w)}{totalWordCount}$$

Steps to compute PMI:

Step 1: Convert it to tokens
Step 2: Count of Words
Step 3: Create Co-occurrence matrix
Step 4: Compute PMI score

Can PMI be negative?

Yes PMI can be negative. Remember log2(0) is -Inf. PMI score lies between $-\infty$ to $+\infty$. For demonstration let's assume that both joint p(w1,w2) and individual p(w1) and p(w2) are 0.001. PMI in that case would be -1. Negative PMI means words are co-occurring less than we expect by chance.

Positive Pointwise Mutual Information (PPMI):

PPMI builds on PMI but addresses some of its limitations, particularly when dealing with sparse datasets (common in NLP). It focuses on emphasizing positive associations and reduces the impact of low-frequency and uninformative word pairs. The formula for PPMI is:

$$PMI(w_1, w_2) = max(\log_2 \frac{p(w_1,w_2)}{p(w_1)p(w_2)}, 0)$$

Information retrieval (IR):

Information retrieval (IR) in natural language processing (NLP) is a field that focuses on the retrieval of relevant information from a large collection of unstructured text data. The primary goal of information retrieval is to help users find the most relevant documents or pieces of information in response to a specific query or information need. Here's an overview of how information retrieval works in NLP:

1. Document Collection: Information retrieval begins with a collection of documents. These documents can be web pages, books, articles, emails, or any other form of text data. This collection is often referred to as a corpus.

2. Query Input: A user or system provides a query in natural language or a set of keywords. This query represents the user's information need, and the goal is to retrieve documents that are most relevant to the query.

3. Preprocessing: Both the query and the document collection undergo preprocessing to make them suitable for retrieval. This preprocessing includes tasks such as tokenization (splitting text into words or tokens), stemming (reducing words to their root form), and removing stop words (common words like "the," "and," "in" that may not be informative).

4. Indexing: To speed up the retrieval process, an index is created. The index contains information about the terms (words) in the documents and their locations within each document. This allows for efficient retrieval of documents containing specific terms.

5. Ranking: Once the index is built, the system ranks the documents in the collection based on their relevance to the query. Various ranking algorithms are used, with common approaches including:

- **Term Frequency-Inverse Document Frequency (TF-IDF):** This method assigns a weight to each term based on its frequency in the document and inverse frequency across the entire corpus. It gives higher importance to terms that are rare in the corpus but frequent in the document.

- **Vector Space Models:** Documents and queries are represented as vectors in a high-dimensional space, and similarity measures (e.g., cosine similarity) are used to rank documents based on their proximity to the query vector.

- **Machine Learning-based Models:** More advanced models, such as neural networks, can be trained to rank documents based on relevance. These models often consider not only term frequencies but also semantic relationships between words.

6. Retrieval: The ranked list of documents is presented to the user, with the most relevant documents appearing at the top. Users can then select and review the documents that are likely to contain the information they need.

7. Evaluation: Information retrieval systems are evaluated using metrics like precision, recall, and F1-score to measure their effectiveness in returning relevant documents and minimizing irrelevant ones.

Information retrieval in NLP is a fundamental component of many applications, including web search engines, document retrieval systems, recommendation systems, and more. It plays a crucial role in helping users access and discover relevant information from vast amounts of textual data.

Relevance Ranking Algorithms:

Relevance ranking algorithms in natural language processing (NLP) are used to determine the order in which documents are presented to users in response to a query. These algorithms aim to rank documents based on their relevance to the query, with the most relevant documents appearing at the top of the search results. Several relevance ranking algorithms are commonly used in NLP:

1. Term Frequency-Inverse Document Frequency (TF-IDF):

- TF-IDF is a statistical measure that evaluates the importance of a term (word) within a document relative to its frequency in a collection of documents (corpus).
- It assigns a weight to each term in a document based on how often it appears in that document (term frequency) and inversely proportional to how often it appears in the entire corpus (inverse document frequency).
- The relevance score for a document is calculated by summing the TF-IDF weights of the query terms present in the document.
- Documents with higher TF-IDF scores for the query terms are considered more relevant.

2. Vector Space Models (VSM):

- VSM represents documents and queries as vectors in a high-dimensional space.
- Each dimension in this space corresponds to a term in the corpus, and the value in each dimension represents the importance of the term in the document or query.
- Cosine similarity is often used to measure the angle between the query vector and the document vectors. Documents with higher cosine similarity values are ranked higher.
- VSM allows for capturing semantic relationships between terms, making it more robust than TF-IDF in some cases.

3. BM25 (Best Matching 25):

- BM25 is a probabilistic retrieval model that builds upon the TF-IDF approach but incorporates some additional adjustments.
- It introduces term saturation and dampening functions to address the limitations of TF-IDF, particularly in longer documents.
- BM25 is known for its effectiveness in handling long documents and is widely used in modern information retrieval systems.

4. Okapi BM25:

- Okapi BM25 is a variant of BM25 that includes further refinements for term weighting and document length normalization.
- It has been successful in a variety of information retrieval applications, including web search engines.

5. Machine Learning-based Models:

- Machine learning algorithms, such as gradient-boosted trees, support vector machines, or neural networks, can be trained to rank documents based on relevance.
- These models take into account various features, including term frequencies, document length, and potentially more complex linguistic or semantic features.
- Learning to rank (LTR) approaches use labelled training data to train models to predict relevance scores.

6. Deep Learning Models:

- Neural networks, particularly deep learning architectures like convolutional neural networks (CNNs) and recurrent neural networks (RNNs), have been applied to relevance ranking tasks.
- These models can capture complex patterns and relationships in text data, making them suitable for tasks where semantic understanding is crucial.

The choice of relevance ranking algorithm often depends on the specific application, the characteristics of the document collection, and the quality of available training data. Many modern information retrieval systems use a combination of these algorithms or employ ensemble techniques to improve ranking performance. Additionally, continuous research in NLP is leading to the development of more advanced and effective ranking models.

Unit 3

Text Preprocessing:

Text preprocessing is a method to clean the text data and make it ready to feed data to the model. Text data contains noise in various forms like emotions, punctuation, text in a different case. When we talk about Human Language then, there are different ways to say the same thing, And this is only the main problem we have to deal with because machines will not understand words, they need numbers so we need to convert text to numbers in an efficient manner.

Techniques:

1. Expand Contractions
2. Lower Case
3. Remove Punctuations
4. Remove words and digits containing digits
5. Remove Stop Words
6. Rephrase Text
7. Stemming and Lemmatization
8. Remove Extra(White) spaces

1. **Expand Contractions:** Contraction is the shortened form of a word like don't stands for do not, aren't stands for are not. Like this, we need to expand this contraction in the text data for better analysis. you can easily get the dictionary of contractions on google or create your own and use the re module to map the contractions.

2. **Lower Case:** If the text is in the same case, it is easy for a machine to interpret the words because the lower case and upper case are treated differently by the machine. for example, words like Ball and ball are treated differently by machine. So, we need to make the text in the same case and the most preferred case is a lower case to avoid such problems.

3. **Remove Punctuations:** One of the other text processing techniques is removing punctuations. There are total 32 main punctuations that need to be taken care of. we can directly use the string module with a regular expression to replace any punctuation in text with an empty string. 32 punctuations which string module provide us is listed below.

4. **Remove Words and Digits Containing Digits:** Sometimes it happens that words and digits combine are written in the text which creates a problem for machines to understand. hence, We need to remove the words and digits which are combined like game57 or game5ts7. This type of word is difficult to process so better to remove them or replace them with an empty string. we use regular expressions for this.

5. **Remove Stop Words:** Stop Words are the most commonly occurring words in a text which do not provide any valuable information. Stop Words like they, there, this, where, etc. are some of the Stop Words. NLTK library is a common library that is used to remove Stop Words and include approximately 180 Stop Words which it removes. If we want to add any new word to a set of words then it is easy using the add method.

6. **Rephrase Text:** We may need to modify some text or change the pattern to a particular string which makes it easy to identify like we can match the pattern of email ids and change it to string like email address.

7. **Stemming and Lemmatization:**

    - **Stemming:** Stemming is a process to reduce the word to its root stem for example run, running, runs, runed derived from the same word as run. Basically, stemming do is remove the prefix or suffix from word like ing, s, es, etc. NLTK library is used to stem the words. The stemming technique is not used for production purposes because it is not so efficient technique and most of the time it stems the unwanted words. So, to solve the problem another technique came into the market as Lemmatization. there are various types of stemming algorithms like porter stemmer, snowball stemmer. Porter stemmer is widely used present in the NLTK library.

    - **Lemmatization:** Lemmatization is similar to stemming, used to stem the words into root word but differs in working. Actually, Lemmatization is a systematic way to reduce the words into their lemma by matching them with a language dictionary.

8. **Remove Extra(White) Spaces:** Most of the time text data contain extra spaces or while performing the above preprocessing techniques more than one space is left between the text so we need to control this problem. regular expression library performs well to solve this problem.

Context-Free Grammars:

What is Grammar?

Grammar is defined as the rules for forming well-structured sentences. Grammar also plays an essential role in describing the syntactic structure of well-formed programs, like denoting the syntactical rules used for conversation in natural languages.

- In the theory of formal languages, grammar is also applicable in Computer Science, mainly in programming languages and data structures. Example - In the C programming language, the precise grammar rules state how functions are made with the help of lists and statements.

- Mathematically, a grammar G can be written as a 4-tuple (N, T, S, P) where:
  - N or VN = set of non-terminal symbols or variables.
  - T or $\sum$ = set of terminal symbols.
  - S = Start symbol where $S \in N$
  - P = Production rules for Terminals as well as Non-terminals.
  - It has the form $\alpha \rightarrow \beta$, where α and β are strings on $\sum V\ N \cup \sum$, and at least one symbol of α belongs to VN

Syntax:

Each natural language has an underlying structure usually referred to under Syntax. The fundamental idea of syntax is that words group together to form the constituents like groups of words or phrases which behave as a single unit.

These constituents can combine to form bigger constituents and, eventually, sentences.

Syntax also refers to the way words are arranged together. Let us see some basic ideas related to syntax:

- **Constituency:** Groups of words may behave as a single unit or phrase - A constituent, for example, like a Noun phrase.

- **Grammatical relations:** These are the formalization of ideas from traditional grammar. Examples include - subjects and objects.

- **Subcategorization and dependency relations:** These are the relations between words and phrases, for example, a Verb followed by an infinitive verb.

- **Regular languages and part of speech:** Refers to the way words are arranged together but cannot support easily. Examples are Constituency, Grammatical relations, and Subcategorization and dependency relations.

- **Syntactic categories and their common denotations in NLP:** np - noun phrase, vp - verb phrase, s - sentence, det - determiner (article), n - noun, tv - transitive verb (takes an object), iv - intransitive verb, prep - preposition, pp - prepositional phrase, adj – adjective

Types of Grammar in NLP:

Let us move on to discuss the types of grammar in NLP. We will cover three types of grammar: context-free, constituency, and dependency.

1. Context Free Grammar: Context-free grammar consists of a set of rules expressing how symbols of the language can be grouped and ordered together and a lexicon of words and symbols.

- One example rule is to express an NP (or noun phrase) that can be composed of either a ProperNoun or a determiner (Det) followed by a Nominal, a Nominal in turn can consist of one or more Nouns: NP → DetNominal, NP → ProperNoun; Nominal → Noun | NominalNoun

- Context-free rules can also be hierarchically embedded, so we can combine the previous rules with others, like the following, that express facts about the lexicon: Det → a Det → the Noun → flight

- Context-free grammar is a formalism power enough to represent complex relations and can be efficiently implemented. Context-free grammar is integrated into many language applications

- A Context free grammar consists of a set of rules or productions, each expressing the ways the symbols of the language can be grouped, and a lexicon of words

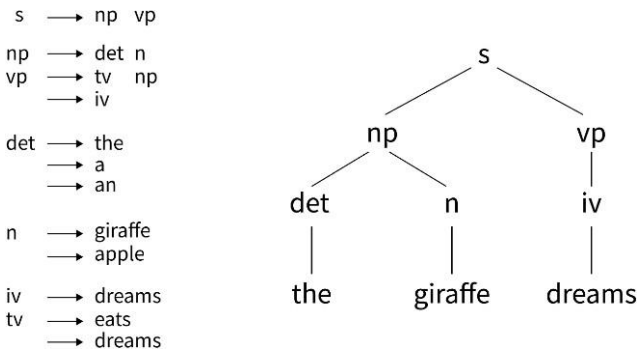
Context-free grammar (CFG) can also be seen as the list of rules that define the set of all well-formed sentences in a language. Each rule has a left-hand side that identifies a syntactic category and a right-hand side that defines its alternative parts reading from left to right. - **Example:** The rule s --> np vp means that "a sentence is defined as a noun phrase followed by a verb phrase."

- **Formalism in rules for context-free grammar:** A sentence in the language defined by a CFG is a series of words that can be derived by systematically applying the rules, beginning with a rule that has s on its left-hand side.

    - Use of parse tree in context-free grammar: A convenient way to describe a parse is to show its parse tree, simply a graphical display of the parse.

    - A parse of the sentence is a series of rule applications in which a syntactic category is replaced by the right-hand side of a rule that has that category on its left-hand side, and the final rule application yields the sentence itself.

- Example: A parse of the sentence "the giraffe dreams" is: s => np vp => det n vp => the n vp => the giraffe vp => the giraffe iv => the giraffe dreams



- If we look at the example parse tree for the sample sentence in the illustration the giraffe dreams, the graphical illustration shows the parse tree for the sentence

- We can see that the root of every subtree has a grammatical category that appears on the left-hand side of a rule, and the children of that root are identical to the elements on the right-hand side of that rule.

Classification of Symbols in CFG:

The symbols used in Context-free grammar are divided into two classes.

- The symbols that correspond to words in the language, for example, the nightclub, are called terminal symbols, and the lexicon is the set of rules that introduce these terminal symbols.

- The symbols that express abstractions over these terminals are called non-terminals.

- In each context-free rule, the item to the right of the arrow (→) is an ordered list of one or more terminals and non-terminals, and to the left of the arrow is a single non-terminal symbol expressing some cluster or generalization. - The non-terminal associated with each word in the lexicon is its lexical category or part of speech.

Context Free Grammar consists of a finite set of grammar rules that have four components: a Set of Non-Terminals, a Set of Terminals, a Set of Productions, and a Start Symbol.

CFG can also be seen as a notation used for describing the languages, a superset of Regular grammar.

CFG consists of a finite set of grammar rules having the following four components:

- **Set of Non-terminals:** It is represented by V. The non-terminals are syntactic variables that denote the sets of strings, which help define the language generated with the help of grammar.
- **Set of Terminals:** It is also known as tokens and is represented by Σ. Strings are formed with the help of the basic symbols of terminals.
- **Set of Productions:** It is represented by P. The set explains how the terminals and non-terminals can be combined.

Every production consists of the following components:

- Non-terminals are also called variables or placeholders as they stand for other symbols, either terminals or non-terminals. They are symbols representing the structure of the language being described. Non-terminals are a set of production rules specifying how to replace a non-terminal symbol with a string of symbols, which can include terminals, words or characters, and other non-terminals.

- **Start Symbol:** The formal language defined by a CFG is the set of strings derivable from the designated start symbol. Each grammar must have one designated start symbol, which is often called S.

- Since context-free grammar is often used to define sentences, S is usually interpreted as the sentence node, and the set of strings that are derivable from S is the set of sentences in some simplified version of English.

Issues with using context-free grammar in NLP:

- **Limited expressiveness:** Context-free grammar is a limited formalism that cannot capture certain linguistic phenomena such as idiomatic expressions, coordination and ellipsis, and even long-distance dependencies.

- **Handling idiomatic expressions:** CFG may also have a hard time handling idiomatic expressions or idioms, phrases whose meaning cannot be inferred from the meanings of the individual words that make up the phrase.

- **Handling coordination:** CFG needs help to handle coordination, which is linking phrases or clauses with a conjunction.

- **Handling ellipsis:** Context-free grammar may need help to handle ellipsis, which is the omission of one or more words from a sentence that is recoverable from the context.

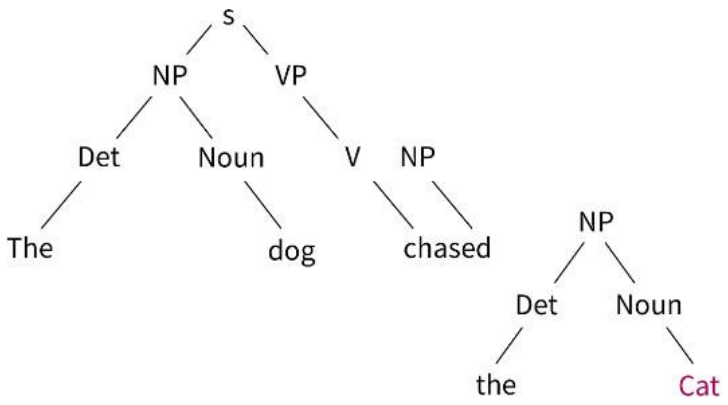
The limitations of context-free grammar can be mitigated by using other formalisms such as dependency grammar which is powerful but more complex to implement, or using a hybrid approach where both constituency and dependency are used together.

2. Constituency Grammar: Constituency Grammar is also known as Phrase structure grammar. Furthermore, it is called constituency Grammar as it is based on the constituency relation. It is the opposite of dependency grammar.

- The constituents can be any word, group of words or phrases in Constituency Grammar. The goal of constituency grammar is to organize any sentence into its constituents using their properties.

- Characteristic properties of constituency grammar and constituency relation:
    - All the related frameworks view the sentence structure in terms of constituency relation.
    - To derive the constituency relation, we take the help of subject-predicate division of Latin as well as Greek grammar.
    - In constituency grammar, we study the clause structure in terms of noun phrase NP and verb phrase VP.

- The properties are derived generally with the help of other NLP concepts like part of speech tagging, a noun or Verb phrase identification, etc. For example, Constituency grammar can organize any sentence into its three constituents - a subject, a context, and an object.



Look at a sample parse tree: Example sentence - "The dog chased the cat."

- In this parse tree, the sentence is represented by the root node S (for sentence). The sentence is divided into two main constituents: NP (noun phrase) and VP (verb phrase).

- The NP is further broken down into Det (determiner) and Noun, and the VP is further broken down into V (verb) and NP.

- Each of these constituents can be further broken down into smaller constituents.

Constituency grammar is better equipped to handle context-free and dependency grammar limitations. Let us look at them:

- Constituency grammar is not language-specific, making it easy to use the same model for multiple languages or switch between languages, hence handling the multilingual issue plaguing the other two types of grammar.

- Since constituency grammar uses a parse tree to represent the hierarchical relationship between the constituents of a sentence, it can be easily understood by humans and is more intuitive than other representation grammars.

- Constituency grammar is robust to errors and can handle noisy or incomplete data.

- Constituency grammar is also better equipped to handle coordination which is the linking of phrases or clauses with a conjunction.

3. Dependency Grammar: Dependency Grammar is the opposite of constituency grammar and is based on the dependency relation. It is opposite to the constituency grammar because it lacks phrasal nodes.

Let us look at some fundamental points about Dependency grammar and dependency relation.

- Dependency Grammar states that words of a sentence are dependent upon other words of the sentence. These Words are connected by directed links in dependency grammar. The verb is considered the center of the clause structure.

- Dependency Grammar organizes the words of a sentence according to their dependencies. Every other syntactic unit is connected to the verb in terms of a directed link. These syntactic units are called dependencies.

- One of the words in a sentence behaves as a root, and all the other words except that word itself are linked directly or indirectly with the root using their dependencies.

- These dependencies represent relationships among the words in a sentence, and dependency grammar is used to infer the structure and semantic dependencies between the words.

Dependency grammar suffers from some limitations; let us understand them further.

- **Ambiguity:** Dependency grammar has issues with ambiguity when it comes to interpreting the grammatical relationships between words, which are particularly challenging when dealing with languages that have rich inflections or complex word order variations.

- **Data annotation:** Dependency parsing also requires labelled data to train the model, which is time-consuming and difficult to obtain.

- **Handling long-distance dependencies:** Dependency parsing also has issues with handling long-term dependencies in some cases where the relationships between words in a sentence may be very far apart, making it difficult to accurately capture the grammatical structure of the sentence.

- **Handling ellipsis and coordination:** Dependency grammar also has a hard time handling phenomena that are not captured by the direct relationships between words, such as ellipsis and coordination, which are typically captured by constituency grammar.

The limitations of dependency grammar can be mitigated by using constituency grammar which, although less powerful, but more intuitive and easier to implement. We can also use a hybrid approach where both constituency and dependency are used together, and it can be beneficial.

Part Of Speech Tagging:

What is Part of Speech (POS) tagging?

Back in elementary school, we have learned the differences between the various parts of speech tags such as nouns, verbs, adjectives, and adverbs. Associating each word in a sentence with a proper POS (part of speech) is known as POS tagging or POS annotation. POS tags are also known as word classes, morphological classes, or lexical tags.

POS tags give a large amount of information about a word and its neighbors. Their applications can be found in various tasks such as information retrieval, parsing, Text to Speech (TTS) applications, information extraction, linguistic research for corpora. They are also used as an intermediate step for higher-level NLP tasks such as parsing, semantics analysis, translation, and many more, which makes POS tagging a necessary function for advanced NLP applications.

Techniques for POS Tagging:

There are various techniques that can be used for POS tagging such as

1. Rule-based POS tagging: The rule-based POS tagging models apply a set of handwritten rules and use contextual information to assign POS tags to words. These rules are often known as context frame rules. One such rule might be: "If an ambiguous/unknown word ends with the suffix 'ing' and is preceded by a Verb, label it as a Verb".

2. Transformation Based Tagging: The transformation-based approaches use a pre- defined set of handcrafted rules as well as automatically induced rules that are generated during training.

3. Deep learning models: Various Deep learning models have been used for POS tagging such as Meta-BiLSTM which have shown an impressive accuracy of around 97 percent.

4. Stochastic (Probabilistic) tagging: A stochastic approach includes frequency, probability, or statistics. The simplest stochastic approach finds out the most frequently used tag for a specific word in the annotated training data and uses this information to tag that word in the unannotated text. But sometimes this approach comes up with sequences of tags for sentences that are not acceptable according to the grammar rules of a language. One such approach is to calculate the probabilities of various tag sequences that are possible for a sentence and assign the POS tags from the sequence with the highest probability. Hidden Markov Models (HMMs) are probabilistic approaches to assign a POS Tag.

1. Rule-based POS Tagging: One of the oldest techniques of tagging is rule-based POS tagging. Rule- based taggers use dictionary or lexicon for getting possible tags for tagging each word. If the word has more than one possible tag, then rule-based taggers use hand-written rules to identify the correct tag. Disambiguation can also be performed in rule-based tagging by analyzing the linguistic features of a word along with its preceding as well as following words. For example, suppose if the preceding word of a word is article then word must be a noun.

As the name suggests, all such kind of information in rule-based POS tagging is coded in the form of rules. These rules may be either:

- Context-pattern rules
- Or, as Regular expression compiled into finite-state automata, intersected with lexically ambiguous sentence representation.

We can also understand Rule-based POS tagging by its two-stage architecture:

- **First Stage:** In the first stage, it uses a dictionary to assign each word a list of potential parts-of-speech.
- **Second Stage:** In the second stage, it uses large lists of hand-written disambiguation rules to sort down the list to a single part-of-speech for each word.

Properties of Rule-Based POS Tagging:

Rule-based POS taggers possess the following properties:

- These taggers are knowledge-driven taggers.
- The rules in Rule-based POS tagging are built manually.
- The information is coded in the form of rules.
- We have some limited number of rules approximately around 1000.
- Smoothing and language modelling is defined explicitly in rule-based taggers.

2. Stochastic POS Tagging: Another technique of tagging is Stochastic POS Tagging. Now, the question that arises here is which model can be stochastic. The model that includes frequency or probability (statistics) can be called stochastic. Any number of different approaches to the problem of part-of-speech tagging can be referred to as stochastic tagger.

The simplest stochastic tagger applies the following approaches for POS tagging:

- **Word Frequency Approach:** In this approach, the stochastic taggers disambiguate the words based on the probability that a word occurs with a particular tag. We can also say that the tag encountered most frequently with the word in the training set is the one assigned to an ambiguous instance of that word. The main issue with this approach is that it may yield inadmissible sequence of tags.
- **Tag Sequence Probabilities:** It is another approach of stochastic tagging, where the tagger calculates the probability of a given sequence of tags occurring. It is also called n-gram approach. It is called so because the best tag for a given word is determined by the probability at which it occurs with the n previous tags.

Properties of Stochastic POST Tagging:

Stochastic POS taggers possess the following properties:

- This POS tagging is based on the probability of tag occurring.
- It requires training corpus
- There would be no probability for the words that do not exist in the corpus.
- It uses different testing corpus (other than training corpus).
- It is the simplest POS tagging because it chooses most frequent tags associated with a word in training corpus.

3. Transformation Based Tagging: Transformation based tagging is also called Brill tagging. It is an instance of the transformation-based learning (TBL), which is a rule-based algorithm for automatic tagging of POS to the given text. TBL, allows us to have linguistic knowledge in a readable form, transforms one state to another state by using transformation rules.

It draws the inspiration from both the previous explained taggers − rule- based and stochastic. If we see similarity between rule-based and transformation tagger, then like rule-based, it is also based on the rules that specify what tags need to be assigned to what words. On the other hand, if we see similarity between stochastic and transformation tagger then like stochastic, it is machine learning technique in which rules are automatically induced from data.

Working of Transformation Based Learning(TBL):

- Start with the solution: The TBL usually starts with some solution to the problem and works in cycles.
- Most beneficial transformation chosen: In each cycle, TBL will choose the most beneficial transformation.
- Apply to the problem: The transformation chosen in the last step will be applied to the problem.

The algorithm will stop when the selected transformation in step 2 will not add either more value or there are no more transformations to be selected. Such kind of learning is best suited in classification tasks.

Advantages of Transformation-based Learning (TBL):

- We learn small set of simple rules and these rules are enough for tagging.
- Development as well as debugging is very easy in TBL because the learned rules are easy to understand.
- Complexity in tagging is reduced because in TBL there is interlacing of machine learned and human-generated rules.
- Transformation-based tagger is much faster than Markov-model tagger.

Disadvantages of Transformation-based Learning (TBL):

- Transformation-based learning (TBL) does not provide tag probabilities.
- In TBL, the training time is very long especially on large corpora.

Hidden Markov Model (HMM) Tagging:

Hidden Markov Model (HMM) is a statistical model that is used to describe the probabilistic relationship between a sequence of observations and a sequence of hidden states. It is often used in situations where the underlying system or process that generates the observations is unknown or hidden, hence it got the name "Hidden Markov Model."

It is used to predict future observations or classify sequences, based on the underlying hidden process that generates the data.

An HMM consists of two types of variables: hidden states and observations.

The hidden states are the underlying variables that generate the observed data, but they are not directly observable.

The observations are the variables that are measured and observed.

The relationship between the hidden states and the observations is modeled using a probability distribution. The Hidden Markov Model (HMM) is the relationship between the hidden states and the observations using two sets of probabilities: the **transition probabilities** and the **emission probabilities**.

The **transition probabilities** describe the probability of transitioning from one hidden state to another.

The **emission probabilities** describe the probability of observing an output given a hidden state.

Hidden Markov Model Algorithm:

The Hidden Markov Model (HMM) algorithm can be implemented using the following steps:

Step 1: Define the state space and observation space: The state space is the set of all possible hidden states, and the observation space is the set of all possible observations.

Step 2: Define the initial state distribution: This is the probability distribution over the initial state.

Step 3: Define the state transition probabilities: These are the probabilities of transitioning from one state to another. This forms the transition matrix, which describes the probability of moving from one state to another.

Step 4: Define the observation likelihoods: These are the probabilities of generating each observation from each state. This forms the emission matrix, which describes the probability of generating each observation from each state.

Step 5: Train the model: The parameters of the state transition probabilities and the observation likelihoods are estimated using the Baum-Welch algorithm, or the forward-backward algorithm. This is done by iteratively updating the parameters until convergence.

Step 6: Decode the most likely sequence of hidden states: Given the observed data, the Viterbi algorithm is used to compute the most likely sequence of hidden states. This can be used to predict future observations, classify sequences, or detect patterns in sequential data.

Step 7: Evaluate the model: The performance of the HMM can be evaluated using various metrics, such as accuracy, precision, recall, or F1 score.

To summarize, the HMM algorithm involves defining the state space, observation space, and the parameters of the state transition probabilities and observation likelihoods, training the model using the Baum-Welch algorithm or the forward-backward algorithm, decoding the most likely sequence of hidden states using the Viterbi algorithm, and evaluating the performance of the model.

POS tagging using an HMM:

1. Understanding Hidden Markov Models (HMMs):

- HMMs are statistical models that assume the existence of hidden states (which are not observed directly) and observable outputs (which are observed).
- In the context of POS tagging, the hidden states represent the POS tags, and the observable outputs are the words in a sentence.

2. Building an HMM for POS Tagging:

- **States (POS tags):** Each POS tag (e.g., noun, verb, adjective) is a hidden state in the model.
- **Observations (words):** The words in a sentence are the observable outputs.
- **Transition probabilities:** HMMs model the transition probabilities between POS tags, i.e., the probability of moving from one POS tag to another in a sequence.

- **Emission probabilities:** HMMs also model the emission probabilities, which are the probabilities of observing a particular word given a specific POS tag.

3. Training the HMM:

- To train an HMM for POS tagging, a corpus with labeled POS tags (annotated sentences) is used.
- The transition probabilities and emission probabilities are estimated from this labeled data. For instance, the frequency of transitions between tags and the frequency of words associated with specific tags are calculated.

4. Decoding and Inference:

- Given a new, unlabeled sentence, the goal is to find the most probable sequence of POS tags for the words in that sentence.
- This involves the use of the Viterbi algorithm, which efficiently finds the most likely sequence of hidden states (POS tags) based on the HMM's transition and emission probabilities.
- The Viterbi algorithm calculates the most probable sequence by considering both the transition probabilities between POS tags and the emission probabilities of observing particular words given specific tags.

5. Tagging the Sentence:

- Finally, the HMM assigns POS tags to each word in the sentence based on the sequence of hidden states (POS tags) obtained from the Viterbi algorithm.

In summary, POS tagging using an HMM involves modeling the probability of sequences of POS tags given the observed words in a sentence, utilizing transition probabilities between POS tags and emission probabilities of words given specific tags to determine the most probable sequence of POS tags for a given sentence.

Conditional Random Fields (CRF):

Conditional Random Fields (CRFs) are a type of probabilistic model used in Natural Language Processing (NLP) for tasks such as sequence labeling, including part-of-speech tagging, named entity recognition, and chunking.

1. Sequential Labeling Problem:

- In NLP, many tasks involve assigning labels to a sequence of inputs (e.g., words in a sentence) where the context and dependencies between labels are important.

2. Modeling with CRFs:

- CRFs are a type of undirected probabilistic graphical model used for structured prediction tasks where the output labels are dependent on each other.
- They model the conditional probability of a sequence of labels given the input sequence, P(Y|X), where Y is the sequence of output labels and X is the input sequence.

3. Features and Dependencies:

- CRFs consider a set of features that capture information from both the input and output sequences.
- Features are typically functions of the input and output, and they capture local or global information about the data.
- CRFs model dependencies among output labels (e.g., neighboring words' POS tags in POS tagging) using these features.

4. Training CRFs:

- Training CRFs involves learning the parameters (weights) associated with the features.
- Optimization techniques, such as gradient descent or other convex optimization methods, are used to estimate these parameters based on labeled training data.
- During training, the model learns which features are relevant and how much influence they have on predicting the output labels.

5. Inference in CRFs:

- Given an input sequence, the task is to find the most probable sequence of output labels.
- CRFs use probabilistic inference algorithms, such as the Viterbi algorithm, to find the best sequence of labels that maximizes the conditional probability P(Y|X).

Advantages of CRFs:

- CRFs handle the dependencies among output labels more effectively compared to simpler models like HMMs.
- They allow for the incorporation of various features, both local and global, making them flexible and adaptable to different types of information present in the data.

Applications in NLP: CRFs have been widely used in various NLP tasks such as named entity recognition, part-of-speech tagging, information extraction, and syntactic parsing due to their ability to model complex dependencies between labels in sequential data.

In summary, CRFs are probabilistic models used for structured prediction in NLP, allowing for the incorporation of features that capture dependencies between input and output sequences to make predictions about sequences of labels in a probabilistic framework.

Named Entity Recognition(NER):

The named entity recognition (NER) is one of the most popular data preprocessing task. It involves the identification of key information in the text and classification into a set of predefined categories. An entity is basically the thing that is consistently talked about or refer to in the text.

NER is the form of NLP.

NLP is just a two-step process, below are the two steps that are involved:

- Detecting the entities from the text
- Classifying them into different categories

Some of the categories that are the most important architecture in NER such that:

- Person
- Organization
- Place/ location

Other common tasks include classifying of the following:

- date/time.
- expression
- Numeral measurement (money, percent, weight, etc.)
- E-mail address

Ambiguity in NE:

- For a person, the category definition is intuitively quite clear, but for computers, there is some ambiguity in classification. Let's look at some ambiguous example:
    - England (Organization) won the 2019 world cup vs The 2019 world cup happened in England(Location).
    - Washington(Location) is the capital of the US vs The first president of the US was Washington(Person).

Methods of NER:

- One way is to train the model for multi-class classification using different machine learning algorithms, but it requires a lot of labelling. In addition to labelling the model also requires a deep understanding of context to deal with the ambiguity of the sentences. This makes it a challenging task for a simple machine learning algorithm.

- Another way is that Conditional random field that is implemented by both NLP Speech Tagger and NLTK. It is a probabilistic model that can be used to model sequential data such as words. The CRF can capture a deep understanding of the context of the sentence. In this model, the input

$$\mathrm{X} = \{\vec{x}_1, \vec{x}_2, \vec{x}_3, \ldots, \vec{x}_T\}$$

$$\mathrm{p}\left(\mathrm{y}—\mathbf{x}\right) = \frac{1}{z(\vec{x})} \prod_{t=1}^{T} \exp\left\{\sum_{k=1}^{k} \omega_k f_k\left(y_t, y_{t-1}, \vec{x}_t\right)\right\}$$

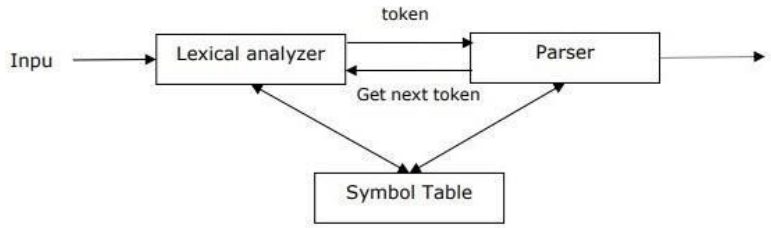
- **Deep Learning Based NER:** deep learning NER is much more accurate than previous method, as it is capable to assemble words. This is due to the fact that it used a method called word embedding, that is capable of understanding the semantic and syntactic relationship between various words. It is also able to learn analyses topic-specific as well as high level words automatically. This makes deep learning NER applicable for performing multiple tasks. Deep learning can do most of the repetitive work itself, hence researchers for example can use their time more efficiently.

Syntactic Analysis:

Syntactic analysis or parsing or syntax analysis is the third phase of NLP. The purpose of this phase is to draw exact meaning, or you can say dictionary meaning from the text. Syntax analysis checks the text for meaningfulness comparing to the rules of formal grammar. For example, the sentence like “hot ice-cream” would be rejected by semantic analyzer.

In this sense, syntactic analysis or parsing may be defined as the process of analyzing the strings of symbols in natural language conforming to the rules of formal grammar. The origin of the word ‘**parsing**’ is from Latin word ‘**pars**’ which means ‘**part**’.

Concept of Parser: It is used to implement the task of parsing. It may be defined as the software component designed for taking input data (text) and giving structural representation of the input after checking for correct syntax as per formal grammar. It also builds a data structure generally in the form of parse tree or abstract syntax tree or other hierarchical structure.



The main roles of the parse include:

- To report any syntax error.
- To recover from commonly occurring error so that the processing of the remainder of program can be continued.
- To create parse tree.
- To create symbol table.
- To produce intermediate representations (IR).

Types of Parsing:

Derivation divides parsing into the followings two types:

- Top-down Parsing
- Bottom-up Parsing

1. Top-down Parsing: In this kind of parsing, the parser starts constructing the parse tree from the start symbol and then tries to transform the start symbol to the input. The most common form of top down parsing uses recursive procedure to process the input. The main disadvantage of recursive descent parsing is backtracking.

2. Bottom-up Parsing: In this kind of parsing, the parser starts with the input symbol and tries to construct the parser tree up to the start symbol.

Concept of Derivation: Derivation is a set of production rules. During parsing, we need to decide the non-terminal, which is to be replaced along with deciding the production rule with the help of which the non-terminal will be replaced.

Types of Derivation:

The two types of derivations, which can be used to decide which non-terminal to be replaced with production rule:

1. Left-most Derivation: In the left-most derivation, the sentential form of an input is scanned and replaced from the left to the right. The sentential form in this case is called the left-sentential form.

2. Right-most Derivation: In the left-most derivation, the sentential form of an input is scanned and replaced from right to left. The sentential form in this case is called the right-sentential form.

Concept of Parse Tree:

It may be defined as the graphical depiction of a derivation. The start symbol of derivation serves as the root of the parse tree. In every parse tree, the leaf nodes are terminals and interior nodes are non-terminals. A property of parse tree is that in-order traversal will produce the original input string.

Concept of Grammar:

A mathematical model of grammar was given by Noam Chomsky in 1956, which is effective for writing computer languages.

Mathematically, a grammar G can be formally written as a 4-tuple (N, T, S, P) where:

- N or VN = set of non-terminal symbols, i.e., variables.
- T or $\sum$ = set of terminal symbols.
- S = Start symbol where $S \in N$
- P denotes the Production rules for Terminals as well as Non-terminals. It has the form $\alpha \rightarrow \beta$, where α and β are strings on $VN \cup \sum$ and least one symbol of α belongs to VN

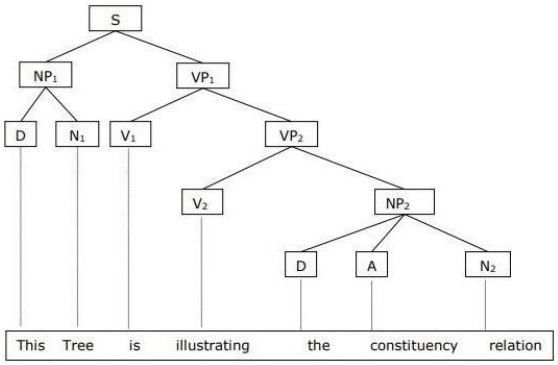
1. Constituency Grammar:

Phrase structure grammar, introduced by Noam Chomsky, is based on the constituency relation. That is why it is also called constituency grammar. It is opposite to dependency grammar.

Example: Before giving an example of constituency grammar, we need to know the fundamental points about constituency grammar and constituency relation.

- All the related frameworks view the sentence structure in terms of constituency relation.
- The constituency relation is derived from the subject-predicate division of Latin as well as Greek grammar.
- The basic clause structure is understood in terms of noun phrase NP and verb phrase VP.

We can write the sentence "**This tree is illustrating the constituency relation**" as follows:



2. Dependency Grammar:

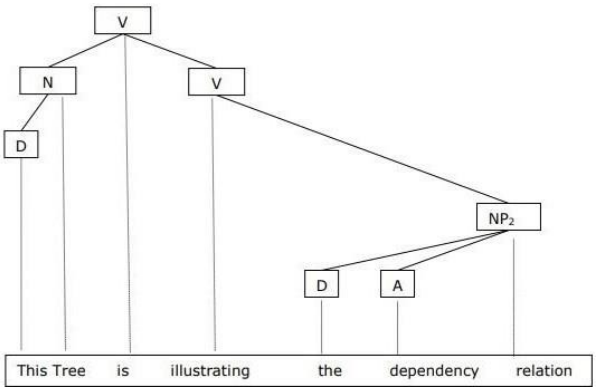
It is opposite to the constituency grammar and based on dependency relation. It was introduced by Lucien Tesniere. Dependency grammar (DG) is opposite to the constituency grammar because it lacks phrasal nodes.

Example:

Before giving an example of Dependency grammar, we need to know the fundamental points about Dependency grammar and Dependency relation.

- In DG, the linguistic units, i.e., words are connected to each other by directed links.
- The verb becomes the center of the clause structure.
- Every other syntactic units are connected to the verb in terms of directed link. These syntactic units are called dependencies.

We can write the sentence "**This tree is illustrating the dependency relation**" as follows:



Parse tree that uses Constituency grammar is called constituency-based parse tree; and the parse trees that uses dependency grammar is called dependency-based parse tree.

Unit 4

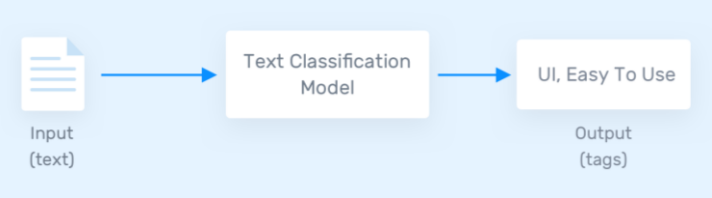
Classifiers for Text Classification and Sentiment Analysis:

What is Text Classification?

Text classification is a machine learning technique that assigns a set of predefined categories to open-ended text. Text classifiers can be used to organize, structure, and categorize pretty much any kind of text – from documents, medical studies and files, and all over the web.

For example, new articles can be organized by topics; support tickets can be organized by urgency; chat conversations can be organized by language; brand mentions can be organized by sentiment; and so on.

Text classification is one of the fundamental tasks in natural language processing with broad applications such as sentiment analysis, topic labelling, spam detection, and intent detection.



Why is Text Classification Important?

It's estimated that around 80% of all information is unstructured, with text being one of the most common types of unstructured data. Because of the messy nature of text, analysing, understanding, organizing, and sorting through text data is hard and time-consuming, so most companies fail to use it to its full potential.

This is where text classification with machine learning comes in. Using text classifiers, companies can automatically structure all manner of relevant text, from emails, legal documents, social media, chatbots, surveys, and more in a fast and cost-effective way. This allows companies to save time analysing text data, automate business processes, and make data-driven business decisions.

Why use machine learning text classification? Some of the top reasons:

- **Scalability:** Manually analysing and organizing is slow and much less accurate.. Machine learning can automatically analyse millions of surveys, comments, emails, etc., at a fraction of the cost, often in just a few minutes. Text classification tools are scalable to any business needs, large or small.
- **Real-time analysis:** There are critical situations that companies need to identify as soon as possible and take immediate action (e.g., PR crises on social media). Machine learning text classification can follow your brand mentions constantly and in real time, so you'll identify critical information and be able to take action right away.
- **Consistent criteria:** Human annotators make mistakes when classifying text data due to distractions, fatigue, and boredom, and human subjectivity creates inconsistent criteria. Machine learning, on the other hand, applies the same lens and criteria to all data and results. Once a text classification model is properly trained it performs with unsurpassed accuracy.

How Does Text Classification Work?

You can perform text classification in two ways: manual or automatic.

Manual text classification involves a human annotator, who interprets the content of text and categorizes it accordingly. This method can deliver good results but it’s time-consuming and expensive.

Automatic text classification applies machine learning, natural language processing (NLP), and other AI-guided techniques to automatically classify text in a faster, more cost-effective, and more accurate manner.

There are many approaches to automatic text classification, but they all fall under three types of systems:

- Rule-based systems
- Machine learning-based systems
- Hybrid systems

1. **Rule-based systems:**

   Rule-based approaches classify text into organized groups by using a set of handcrafted linguistic rules. These rules instruct the system to use semantically relevant elements of a text to identify relevant categories based on its content. Each rule consists of an antecedent or pattern and a predicted category.

   Say that you want to classify news articles into two groups: *Sports* and *Politics*. First, you'll need to define two lists of words that characterize each group (e.g., words related to sports such as *football*, *basketball*, *LeBron James*, etc., and words related to politics, such as *Donald Trump*, *Hillary Clinton*, *Putin*, etc.).

   Next, when you want to classify a new incoming text, you'll need to count the number of sport-related words that appear in the text and do the same for politics-related words. If the number of sports-related word appearances is greater than the politics-related word count, then the text is classified as Sports and vice versa.

   For example, this rule-based system will classify the headline *"When is LeBron James' first game with the Lakers?"* as *Sports* because it counted one sports-related term (LeBron James) and it didn't count any politics-related terms.

   Rule-based systems are human comprehensible and can be improved over time. But this approach has some disadvantages. For starters, these systems require deep knowledge of the domain. They are also time-consuming, since generating rules for a complex system can be quite challenging and usually requires a lot of analysis and testing. Rule-based systems are also difficult to maintain and don't scale well given that adding new rules can affect the results of the pre-existing rules.

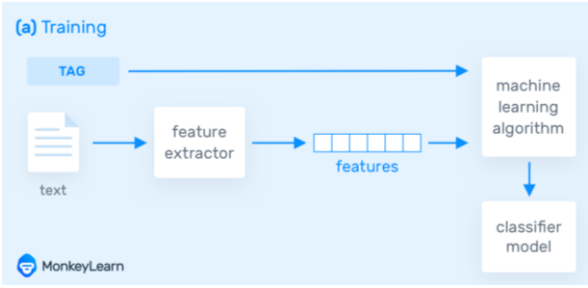
2. Machine learning based systems:

Instead of relying on manually crafted rules, machine learning text classification learns to make classifications based on past observations. By using pre-labelled examples as training data, machine learning algorithms can learn the different associations between pieces of text, and that a particular output (i.e., tags) is expected for a particular input (i.e., text). A "tag" is the pre-determined classification or category that any given text could fall into.

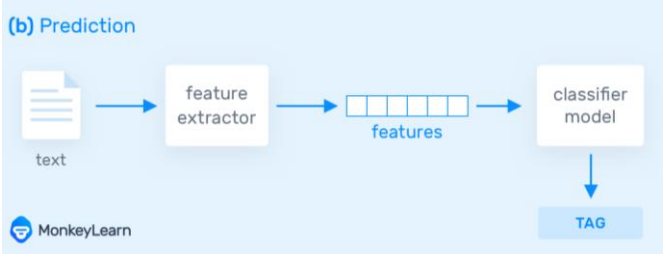
The first step towards training a machine learning NLP classifier is feature extraction: a method is used to transform each text into a numerical representation in the form of a vector. One of the most frequently used approaches is bag of words, where a vector represents the frequency of a word in a predefined dictionary of words.

For example, if we have defined our dictionary to have the following words {*This, is, the, not, awesome, bad, basketball*}, and we wanted to vectorize the text *"This is awesome,"* we would have the following vector representation of that text: (1, 1, 0, 0, 1, 0, 0).

Then, the machine learning algorithm is fed with training data that consists of pairs of feature sets (vectors for each text example) and tags (e.g., *sports*, *politics*) to produce a classification model:



Once it's trained with enough training samples, the machine learning model can begin to make accurate predictions. The same feature extractor is used to transform unseen text to feature sets, which can be fed into the classification model to get predictions on tags (e.g., *sports*, *politics*):



Text classification with machine learning is usually much more accurate than human-crafted rule systems, especially on complex NLP classification tasks. Also, classifiers with machine learning are easier to maintain and you can always tag new examples to learn new tasks.

Machine Learning Text Classification Algorithms:

Some of the most popular text classification algorithms include the Naive Bayes family of algorithms, support vector machines (SVM), and deep learning.

1. **Naive Bayes:**

   The Naive Bayes family of statistical algorithms are some of the most used algorithms in text classification and text analysis, overall.

   One of the members of that family is Multinomial Naive Bayes (MNB) with a huge advantage, that you can get really good results even when your dataset isn't very large (~ a couple of thousand tagged samples) and computational resources are scarce.

   Naive Bayes is based on Bayes's Theorem, which helps us compute the conditional probabilities of the occurrence of two events, based on the probabilities of the occurrence of each individual event. So, we're calculating the probability of each tag for a given text, and then outputting the tag with the highest probability.

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

   The probability of A, if B is true, is equal to the probability of B, if A is true, times the probability of A being true, divided by the probability of B being true.

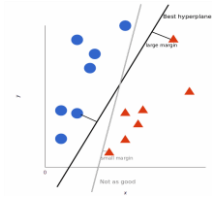
   This means that any vector that represents a text will have to contain information about the probabilities of the appearance of certain words within the texts of a given category, so that the algorithm can compute the likelihood of that text belonging to the category.

2. Support Vector Machines:

Support Vector Machines (SVM) is another powerful text classification machine learning algorithm, because Naive Bayes, SVM doesn't need much training data to start providing accurate results. SVM does, however, require more computational resources than Naive Bayes, but the results are even faster and more accurate.

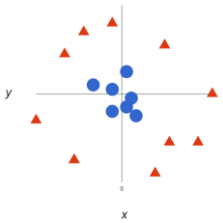
In short, SVM draws a line or "hyperplane" that divides a space into two subspaces. One subspace contains vectors (tags) that belong to a group, and another subspace contains vectors that do not belong to that group.

The optimal hyperplane is the one with the largest distance between each tag. In two dimensions it looks like this:



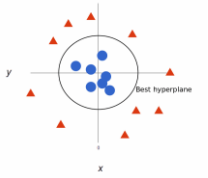
Those vectors are representations of your training texts, and a group is a tag you have tagged your texts with.

As data gets more complex, it may not be possible to classify vectors/tags into only two categories. So, it looks like this:



But that's the great thing about SVM algorithms – they're "multi-dimensional." So, the more complex the data, the more accurate the results will be. Imagine the above in three dimensions, with an added Z-axis, to create a circle.

Mapped back to two dimensions the ideal hyperplane looks like this:



3. Deep Learning:

Deep learning is a set of algorithms and techniques inspired by how the human brain works, called neural networks. Deep learning architectures offer huge benefits for text classification because they perform at super high accuracy with lower-level engineering and computation.

The two main deep learning architectures for text classification are Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN).

Deep learning is hierarchical machine learning, using multiple algorithms in a progressive chain of events. It's similar to how the human brain works when making decisions, using different techniques simultaneously to process huge amounts of data.

Deep learning algorithms do require much more training data than traditional machine learning algorithms (at least millions of tagged examples). However, they don't have a threshold for learning from training data, like traditional machine learning algorithms, such as SVM and NBeep learning classifiers continue to get better the more data you feed them with:

Deep learning algorithms, like Word2Vec or GloVe are also used in order to obtain better vector representations for words and improve the accuracy of classifiers trained with traditional machine learning algorithms.

4. Hybrid Systems:

Hybrid systems combine a machine learning-trained base classifier with a rule-based system, used to further improve the results. These hybrid systems can be easily fine-tuned by adding specific rules for those conflicting tags that haven't been correctly modelled by the base classifier.

5. Metrics and Evaluation:

Cross-validation is a common method to evaluate the performance of a text classifier. It works by splitting the training dataset into random, equal-length example sets (e.g., 4 sets with 25% of the data). For each set, a text classifier is trained with the remaining samples (e.g., 75% of the samples). Next, the classifiers make predictions on their respective sets, and the results are compared against the human-annotated tags. This will determine when a prediction was right (true positives and true negatives) and when it made a mistake (false positives, false negatives).

With these results, you can build performance metrics that are useful for a quick assessment on how well a classifier works:

- **Accuracy:** the percentage of texts that were categorized with the correct tag.
- **Precision:** the percentage of examples the classifier got right out of the total number of examples that it predicted for a given tag.
- **Recall:** the percentage of examples the classifier predicted for a given tag out of the total number of examples it should have predicted for that given tag.
- **F1 Score:** the harmonic mean of precision and recall.

Text Classification Applications & Use Cases:

Text classification has thousands of use cases and is applied to a wide range of tasks. In some cases, data classification tools work behind the scenes to enhance app features we interact with on a daily basis (like email spam filtering). In some other cases, classifiers are used by marketers, product managers, engineers, and salespeople to automate business processes and save hundreds of hours of manual data processing.

Some of the top applications and use cases of text classification include:

- Detecting urgent issues
- Automating customer support processes
- Listening to the Voice of customer (VoC)

1. **Detecting Urgent Issues:**

   On Twitter alone, users send 500 million tweets every day.

   And surveys show that 83% of customers who comment or complain on social media expect a response the same day, with 18% expecting it to come immediately.

   With the help of text classification, businesses can make sense of large amounts of data using techniques like aspect-based sentiment analysis to understand what people are talking about and how they're talking about each aspect. For example, a potential PR crisis, a customer that's about to churn, complaints about a bug issue or downtime affecting more than a handful of customers.

2. **Automating Customer Support Processes:**

   Building a good customer experience is one of the foundations of a sustainable and growing company. According to Hubspot, people are 93% more likely to be repeat customers at companies with excellent customer service. The study also unveiled that 80% of respondents said they had stopped doing business with a company because of a poor customer experience.

   Text classification can help support teams provide a stellar experience by automating tasks that are better left to computers, saving precious time that can be spent on more important things.

   For instance, text classification is often used for automating ticket routing and triaging. Text classification allows you to automatically route support tickets to a teammate with specific product expertise. If a customer writes in asking about refunds, you can automatically assign the ticket to the teammate with permission to perform refunds. This will ensure the customer gets a quality response more quickly.

   Support teams can also use sentiment classification to automatically detect the urgency of a support ticket and prioritize those that contain negative sentiments. This can help you lower customer churn and even turn a bad situation around.

3. Listening to Voice of Customer (VoC):

Companies leverage surveys such as Net Promoter Score to listen to the voice of their customers at every stage of the journey.

The information gathered is both qualitative and quantitative, and while NPS scores are easy to analyse, open-ended responses require a more in-depth analysis using text classification techniques. Instead of relying on humans to analyse voice of customer data, you can quickly process open-ended customer feedback with machine learning. Classification models can help you analyse survey results to discover patterns and insights like:

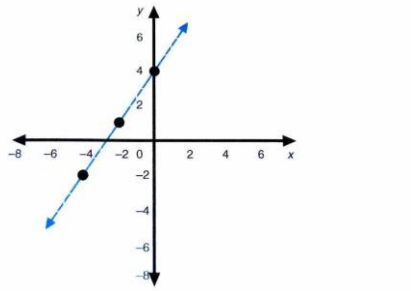
- What do people like about our product or service?
- What should we improve?
- What do we need to change?

By combining both quantitative results and qualitative analyses, teams can make more informed decisions without having to spend hours manually analysing every single open-ended response.

Text Classification with Logistic Regression Model:

Text classification is a fundamental problem in natural language processing (NLP) that involves categorising text data into predefined classes or categories. It can be used in many real-world situations, like sentiment analysis, spam filtering, topic modelling, and content classification, to name a few.

While logistic regression has some limitations, such as the assumption of a linear relationship between the input features and the class labels, it remains a useful and practical approach to text classification.



Logistic regression assumes a linear relationship between the input features and the class labels.

Why use logistic regression?

Logistic regression is a popular algorithm for text classification and is also our go-to favourite for several reasons:

1. **Simplicity:** Logistic regression is a relatively simple algorithm that is easy to implement and interpret. It can be trained efficiently even on large datasets, making it a practical choice for many real-world applications.
2. **Easily understood:** Logistic regression models can be understood by looking at the coefficients of the input features, which can show which words or phrases are most important for classification.
3. **Works well with sparse data:** Text data is often very high-dimensional and sparse, meaning many features are zero for most data points. Logistic regression can handle sparse data well and can be regularised to prevent overfitting.
4. **Versatile:** Logistic regression works well for both binary and multi-class classification. It is a versatile algorithm for text classification that can be used for binary and multi-class classification tasks.

5. **Baseline model:** Logistic regression can be used as a baseline model for classifying text. This lets you compare how well more complicated algorithms work with a simple model that is easy to understand.

Logistic regression is a practical algorithm for classifying text that can give good results in many situations, especially for more straightforward classification tasks or as a starting point for more complicated algorithms.

How to use logistic regression for text classification:

Logistic regression is a commonly used statistical method for binary classification tasks, including text classification.

In text classification, the goal is to assign a given piece of text to one or more predefined categories or classes.

To use logistic regression for text classification, we first need to represent the text as numerical features that can be used as input to the model. One popular approach for this is to use the bag-of-words representation, where we represent each document as a vector of word frequencies.

Once we have our numerical feature representation of the text, we can use logistic regression to learn a model to predict the probability of each document belonging to a given class. The logistic regression model learns a set of weights for each feature and uses these weights to make predictions based on the input features.

During training, we adjust the weights to minimise a loss function, such as cross-entropy, that measures the difference between the predicted probabilities and the actual labels. Once the model is trained, we can use it to predict the class labels for new text inputs.

Overall, logistic regression is a simple but effective method for text classification tasks and can be used as a baseline model or combined with more complex models in ensemble approaches. However, it may need help with more complex relationships between features and labels and may not capture the full range of patterns in natural language data.

Multinomial Logistic Regression:

Multinomial Logistic Regression is a statistical technique used for modelling relationships between multiple categories of a dependent variable and one or more independent variables. In the context of Natural Language Processing (NLP), it's often used for tasks like text classification or sentiment analysis where there are multiple classes to predict.

1. Basic Logistic Regression:

- Logistic Regression is a binary classification algorithm used to model the probability of a binary outcome given a set of independent variables.
- It models the log-odds of the probability of an event occurring.

2. Extension to Multiple Classes:

- Multinomial Logistic Regression is an extension of logistic regression that accommodates multiple classes (more than two) in the dependent variable.
- Instead of predicting just two classes as in binary logistic regression, it can handle more than two categories simultaneously.

3. Probability Distribution:

- In Multinomial Logistic Regression, the probabilities of the multiple classes are modelled using the multinomial distribution.
- The model estimates the probabilities of each class as a function of the independent variables.

4. Mathematical Representation:

- Given a set of input features X and multiple classes C_1, C_2,…C_k, Multinomial Logistic Regression calculates the probabilities for each class using a SoftMax function.
- The SoftMax function calculates the probability of each class as the exponential of the linear combination of features divided by the sum of exponentials of all classes' linear combinations.

5. Training and Optimization:

- During training, the model learns the weights (coefficients) associated with each feature for each class.
- Optimization techniques such as gradient descent or variants are used to find the optimal weights that minimize the error between predicted probabilities and actual classes.

6. Prediction:

- To predict the class of a new instance, the model calculates the probabilities for each class using the learned weights and the input features.
- The class with the highest probability is assigned as the predicted class for the instance.

Applications in NLP: Multinomial Logistic Regression can be used for tasks like text classification (e.g., sentiment analysis, topic classification), where there are multiple classes to predict based on text features.

In summary, Multinomial Logistic Regression is a classification algorithm that extends binary logistic regression to handle multiple classes. It models the probabilities of multiple classes using a multinomial distribution and estimates the relationship between input features and multiple classes in a probabilistic framework.

Unit 5

Simple Recurrent Networks:

Simple Recurrent Networks (SRNs) are a type of recurrent neural network (RNN) designed to process sequential data by maintaining a form of memory or context across time steps. They are considered the most basic form of RNNs. SRNs have connections that form a directed cycle, allowing information to persist and be passed from one time step to the next within the network.

Key components and how SRNs function:

1. **Recurrent Connections:** SRNs have feedback connections that allow information to loop back into the network at the next time step. This loop enables the network to maintain a memory of past inputs, making it suitable for sequential data where past context matters.
2. **Processing at Each Time Step:** At each time step, the network takes an input and the previous hidden state (representing past information) and processes them to produce an output and a new hidden state. This hidden state acts as the memory or context that is updated with each new input.
3. **Training:** SRNs are trained using techniques like backpropagation through time (BPTT), which is an extension of the backpropagation algorithm. BPTT unfolds the network through time steps, treating it as a deep feedforward neural network, and computes gradients to update the network's parameters.
4. **Challenges:** Simple Recurrent Networks face issues like vanishing or exploding gradients, where the signal propagated through the recurrent connections either diminishes exponentially or grows uncontrollably during training, affecting the network's ability to learn long-range dependencies.

Despite their conceptual simplicity, SRNs have limitations in capturing long-term dependencies in sequences due to these gradient-related issues. More advanced RNN architectures, like Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs), were developed to address these problems and better capture long-range dependencies by incorporating specialized memory mechanisms.

SRNs serve as the foundation for understanding the principles behind recurrent neural networks, providing insights into how information flows across time steps within sequential data processing.

Applications of Recurrent Neural Networks:

- Prediction problems
- Machine Translation
- Speech Recognition
- Language Modelling and Generating Text
- Video Tagging
- Generating Image Descriptions
- Text Summarization
- Call Centre Analysis
- Face detection,
- OCR Applications as Image Recognition

Here we will discuss a few of the projects:

- Sentiment Classification
- From the name itself, we can understand that to identify the sentiment based on the review.
- The task is to simply classify the tweets into positive and negative sentiment. Here the input which tweets can have various lengths. But in Recurrent neural network, we always have an output with the same length of the input.

Image Captioning: Image captioning is a very interesting project where you will have an image and for that particular image, you need to generate a textual description.

So here,

1. The input will be single input – the image,
2. And the output will be a series or sequence of words

Here the image might be of a fixed size, but the description will vary for the length.

Language Translation: Language Translation is an application which we use almost every day in our life. We are all quite familiar with Google Lense where you can just convert one language to another one using a lens. So this the application of language translation

Suppose you have some text in a particular language. Let's assume English, and you don't know English so you want to translate them into French. So that time we used a language translator.

Deep Networks:

Stacked and Bidirectional RNNs:

Bidirectional recurrent neural networks (Bidirectional RNNs) are artificial neural networks that process input data in both the forward and backward directions. Bidirectional recurrent neural networks are really just putting two independent RNNs together. It consists of two separate RNNs that process the input data in opposite directions, and the outputs of these RNNs are combined to produce the final output. One common way to combine the outputs of the forward and reverse RNNs is to concatenate them, but other methods, such as element-wise addition or multiplication can also be used. The choice of combination method can depend on the specific task and the desired properties of the final output.

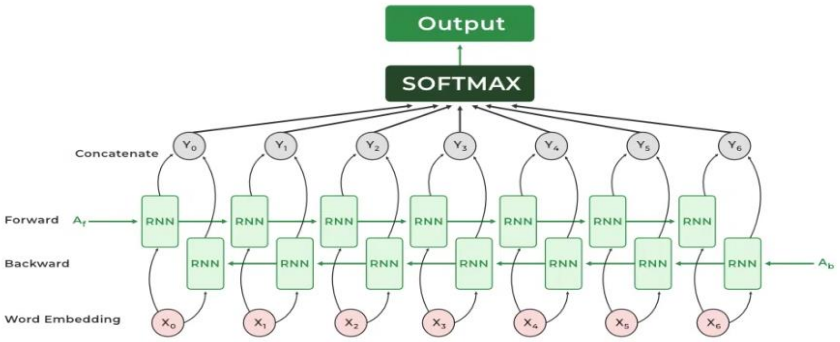
They are often used in natural language processing tasks, such as language translation, text classification, and named entity recognition. They can capture contextual dependencies in the input data by considering past and future contexts.

Need for Bidirectional Recurrent Neural Networks:

- Bidirectional Recurrent Neural Networks (RNNs) are used when the output at a particular time step depends on the input at that time step as well as the inputs that come after it. However, in some cases, the output at a particular time step may also depend on the inputs that come before it. In such cases, Bidirectional RNNs are used to capture the dependencies in both directions.
- The main need for Bidirectional RNNs arises in sequential data processing tasks where the context of the data is important. For instance, in natural language processing, the meaning of a word in a sentence may depend on the words that come before and after it. Similarly, in speech recognition, the current sound may depend on the previous and upcoming sounds.
- The need for Bidirectional RNNs arises in tasks where the context of the data is important, and the output at a particular time step depends on both past and future inputs. By processing the input sequence in both directions, Bidirectional RNNs help to capture these dependencies and improve the accuracy of predictions.

The architecture of Bidirectional RNN:

A Bidirectional RNN is a combination of two RNNs – one RNN moves forward, beginning from the start of the data sequence, and the other, moves backward, beginning from the end of the data sequence. The network blocks in a Bidirectional RNN can either be simple RNNs, GRUs, or LSTMs.



Bidirectional Recurrent Neural Networks

A Bidirectional RNN has an additional hidden layer to accommodate the backward training process. At any given time t, the forward and backward hidden states are updated as follows:

$A_t(\text{forward}) = \varphi(X_t \ W_{Xf} + A_{t-1}(\text{forward}) \ W_{Af} + b_{Af})$

$A_t(\text{backward}) = \varphi(X_t \ W_{Xb} + A_{t-1}(\text{backward}) \ W_{Ab} + b_{Ab})$

where φ is the activation function, W is the weight matrix, and b is the bias.

The final hidden state is the concatenation of A_t(forward) and A_t(backward)

$$A_t = [A_t(\text{forward});A_t(\text{backward})]$$

OR

$$A_t = A_t(\text{forward}) \oplus A_t(\text{backward})$$

Here, \oplus denotes the mean vector concatenation. There are some other ways also to combine both forward and backward hidden states like element-wise addition or multiplication.

The hidden state at time t is given by the combination of A_t(forward) and A_t(backward). The output of any given hidden state is given by:

$$y_t = A_t * W_{Ay} + b_y$$

$$p_t = \text{softmax}(y_t)$$

The training of a Bidirectional RNN is similar to **Backpropagation Through Time (BPTT)** algorithm. BPTT is the backpropagation algorithm used while training RNNs. A typical BPTT algorithm works as follows:
Unroll the network and compute errors at every time step.

$$L = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} o_{i,c}\log(p_{i,c})$$

Here,

- N = Number of samples
- C = Number of classes
- $o_{i,c}$ is the ground truth label for the i^{th} sample and c^{th} class. It is a one-hot encoded vector with a value of 1 for the true class and 0 for the other classes.
- $p_{i,c}$ is the predicted probability for the i^{th} sample and c^{th} class. It is a one-hot encoded vector with a value of 1 for the true class and 0 for the other classes. , which is output by the last layer of the BiRNN. It is a vector of predicted probabilities for each class.

Roll up the network and update weights:

In a Bidirectional RNN however, since there are forward and backward passes happening simultaneously, updating the weights for the two processes could happen at the same point in time. This leads to erroneous results. Thus, to accommodate forward and backward passes separately, the following algorithm is used for training a Bidirectional RNN:

Forward Pass

- Forward states (from t = 1 to N) and backward states (from t = N to 1) are passed.
- Output neuron values are passed (from t = 1 to N)

Backward Pass

- Output neuron values are passed (from t = N to 1)
- Forward states (from t = N to 1) and backward states (from t = 1 to N) are passed.

Both the forward and backward passes together train a Bidirectional RNN.

Explanation of Bidirectional RNN with a simple example:

Traditional RNNs like GRUs and LSTMs grasp context only from preceding words, unable to anticipate future ones. Bidirectional RNNs tackle this by processing sequences in both directions, using two RNNs. Their hidden states merge into a single one for decoding—this could be either the whole sequence or the last time step's state, impacting the neural network's design.

Managing Context in RNNs:

LSTMs and GRUs:

Problems with RNN :

Exploding and vanishing gradient problems during backpropagation.

Gradients are those values which to update neural networks weights. In other words, we can say that Gradient carries information.

Vanishing gradient is a big problem in deep neural networks. it vanishes or explodes quickly in earlier layers and this makes RNN unable to hold information of longer sequence. and thus **RNN becomes short-term memory**.

If we apply RNN for a paragraph RNN may leave out necessary information due to gradient problems and not be able to carry information from the initial time step to later time steps.

To solve this problem LSTM, GRU came into the picture.

How do LSTM, GRU solve this problem?

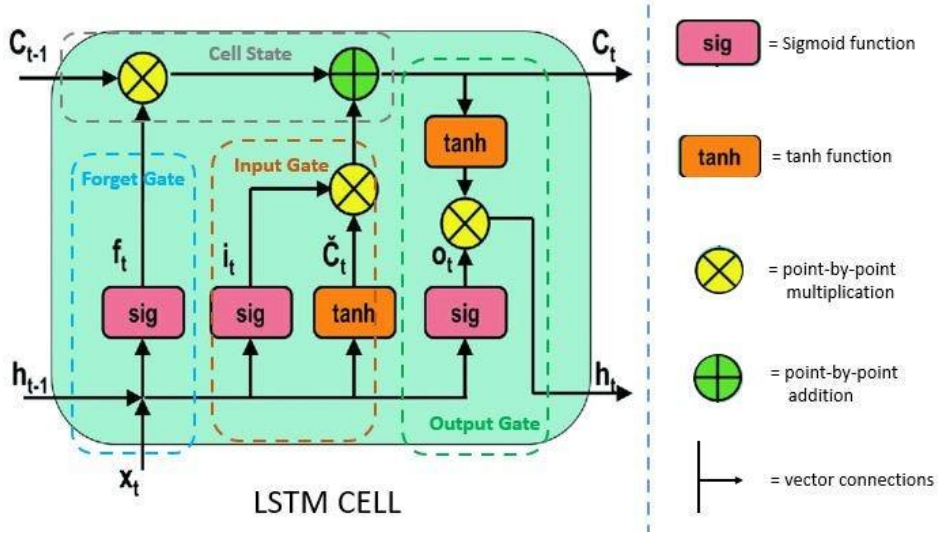
The reason for exploding gradient was the capturing of relevant and irrelevant information. a model which can decide what information from a paragraph and relevant and remember only relevant information and throw all the irrelevant information.

This is achieved by using gates. the LSTM (Long -short-term memory) and GRU (Gated Recurrent Unit) have gates as an internal mechanism, which control what information to keep and what information to throw out. By doing this LSTM, GRU networks solve the exploding and vanishing gradient problem.

Almost each and every SOTA (state of the art) model based on RNN follows LSTM or GRU networks for prediction.
LSTMs /GRUs are implemented in speech recognition, text generation, caption generation, etc.

LSTM networks:

Every LSTM network basically contains three gates to control the flow of information and cells to hold information. The Cell States carries the information from initial to later time steps without getting vanished.



Gates

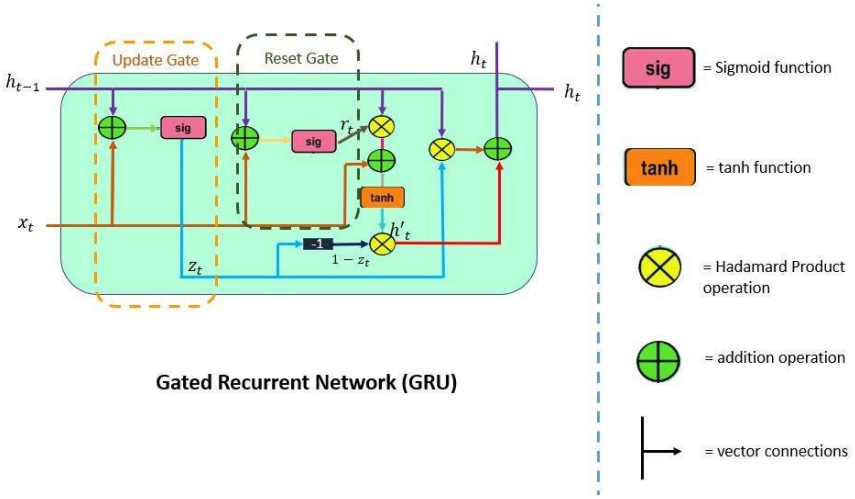
Gates make use of sigmoid activation or you can say *tanh* activation. values ranges in *tanh* activation are 0 -1.

1. Forget Gate
2. Input Gate
3. Output Gate

Let's see these gates in detail:

1. **Forget Gate:** This gate decides what information should be carried out forward or what information should be ignored.
   Information from previous hidden states and the current state information passes through the sigmoid function. Values that come out from sigmoid are always between 0 and 1. if the value is closer to 1 means information should proceed forward and if value closer to 0 means information should be ignored.
2. **Input Gate**: After deciding the relevant information, the information goes to the input gate, Input gate passes the relevant information, and this leads to updating the cell states. simply saving updating the weight.
   Input gate adds the new relevant information to the existing information by updating cell states.
3. **Output Gate**: After the information is passed through the input gate, now the output gate comes into play. Output gate generates the next hidden states. and cell states are carried over the next time step.

GRU:



Gated Recurrent Network (GRU)

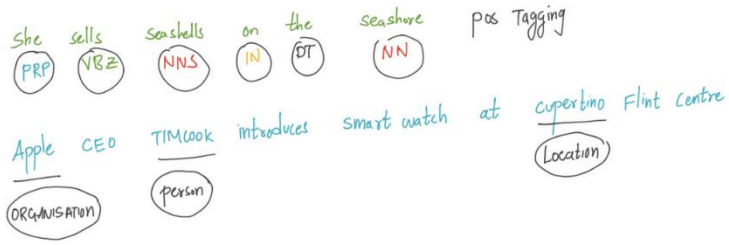
GRU (Gated Recurrent Units) are similar to the LSTM networks. GRU is a kind of newer version of RNN. However, there are some differences between GRU and LSTM.

- GRU doesn't contain a cell state
- GRU uses its hidden states to transport information
- It Contains only 2 gates(Reset and Update Gate)
- GRU is faster than LSTM
- GRU has lesser tensor's operation that makes it faster

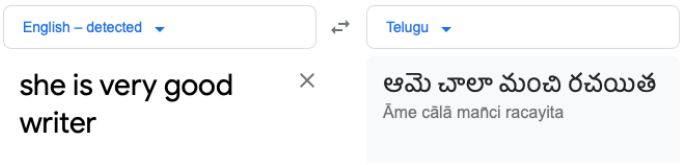
1. **Update Gate:** Update Gate is a combination of Forget Gate and Input Gate. Forget gate decides what information to ignore and what information to add in memory.
2. **Reset Gate:** This Gate Resets the past information in order to get rid of gradient explosion. Reset Gate determines how much past information should be forgotten.

The Encoder-Decoder Model with RNNs:

In tasks like machine translation, we must map from a sequence of input words to a sequence of output words. The reader must note that this is not similar to "sequence labelling", where that task it to map each word in the sequence to a predefined classes, like part-of-speech or named entity task.



In above two examples, the models are tasked to map each word in the sequence to a tag/class.



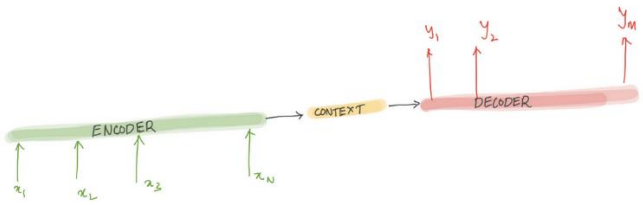
Google translation

But in tasks like machine translation: the length of inputs sequence need to not necessarily length of output sequence. As you can see in the google translation example, the input length is "5" and output length is "4". Since we are mapping an input sequence to an output sequence, thus comes the name **sequence to sequence models.** Not only the length of input and output sequence differs but the order of words also differ. This is very complex task in NLP and Encoder- decoder networks are very successful at handling these sorts of complicated tasks of sequence-to-sequence mapping.

One more important task that can be solved with encoder-decoder networks is text summarisation where we map the long text to a short summary/abstract. In this blog we will try to understand the architecture of encoder-decoder networks and how it works.

The Encoder-Decoder Network:

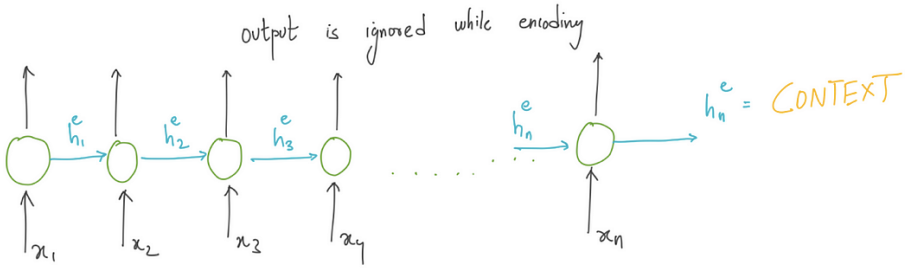
This network have been applied to very wide range of applications including machine translation, text summarisation, questioning answering and dialogue. Let's try to understand the idea underlying the encoder-decoder networks. The encoder takes the input sequence and creates a contextual representation (which is also called context) of it and the decoder takes this contextual representation as input and generates output sequence.



Encoder and Decoder with RNN's:

All variants of RNN's can be employed as encoders and decoders . In RNN's we have notion of hidden state "**ht**" which can be seen as a summary of words/tokens it has seen till time step "**t**" in the sequence chain.

Encoder: Encoder takes the input sequence and generated a context which is the essence of the input to the decoder.

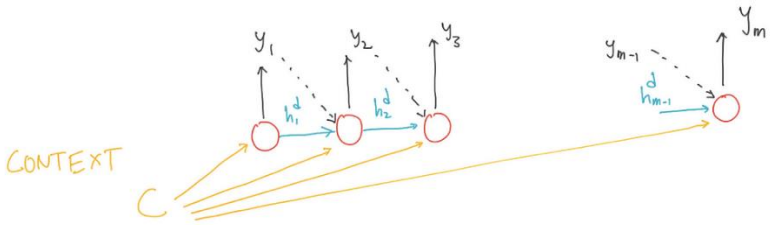


Using RNN as encoder

The entire purpose of the encoder is to generate a contextual representation/ context for the input sequence. Using RNN as encoder, the final hidden state of the RNN sequence chain can be used a proxy for context.

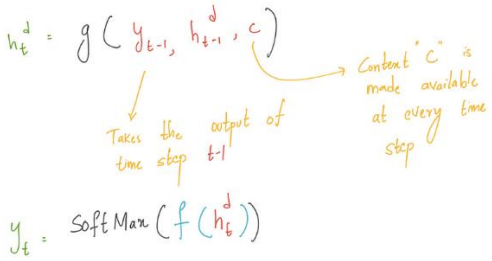
This is the most critical concept which forms the basis for encoder-decoder models. We will use the subscripts e and d for the hidden state of the encoder and decoder. Outputs of encoder is ignored, as the goal is to generate final hidden state or context for decoder.

Decoder: Decoder takes the context as input and generates a sequence of output. When we employ RNN as decoder, the context is the final hidden state of the RNN encoder.



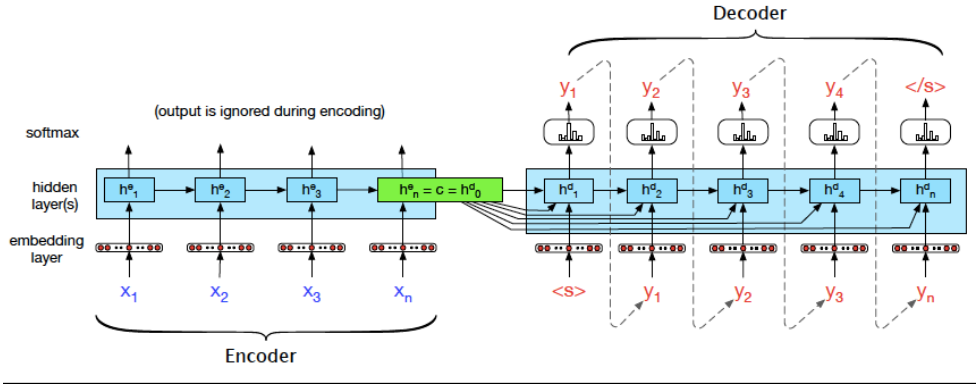
The first decoder RNN cell takes "CONTEXT" as its prior hidden state. The decoder then generated the output until the end-of-sequence marker is generated.

Each cell in RNN decoder takes input auto regressively, i.e., The decoder uses its own estimated output at time t as the input for the next time step xt+1. One important drawback if the context is made available only for first decoder RNN cell is the context wanes as more and more output sequence is generated. To overcome this drawback the "CONTEXT" can be made available at each decoding RNN time step. There is a little deviation from the vanilla-RNN. Let's look at the updated the equations for decoder RNN.



Updated Equations for RNN decoder

Training the Encoder — Decoder Model:



Complete Encoder and Decoder network

The training data consists of sets of input sentences and their respective output sequences.

We use cross entropy loss in the decoder. Encoder-decoder architectures are trained end-to-end, just as with the RNN language models. The loss is calculated and then back-propagated to update weights using the gradient descent optimisation. The total loss is calculated by averaging the cross-entropy loss per target word.

Transformers and Pretrained Language Models:

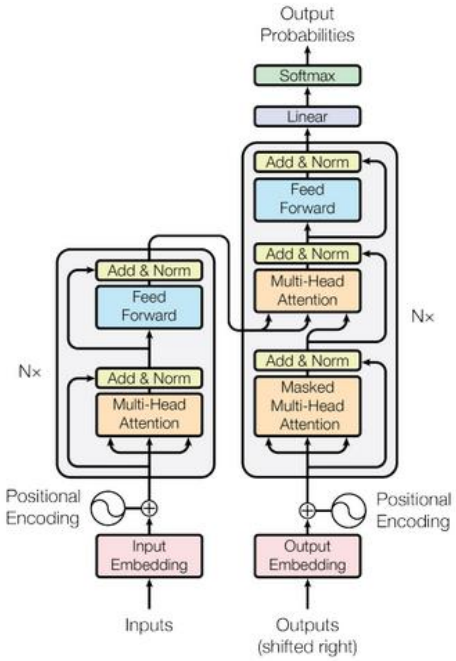
Transformer:

The Transformer in NLP is a novel architecture that aims to solve sequence-to-sequence tasks while handling long-range dependencies with ease. The Transformer was proposed in the paper Attention Is All You Need. It is recommended reading for anyone interested in NLP.

Here, "transduction" means the conversion of input sequences into output sequences. The idea behind Transformer is to handle the dependencies between input and output with **attention** and recurrence completely.

Let's take a look at the architecture of the Transformer below. It might look intimidating but don't worry, we will break it down and understand it block by block.

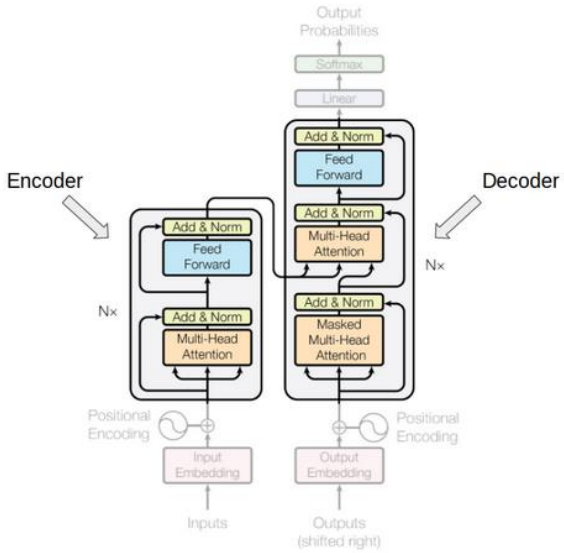
Understanding Transformer's Model Architecture:



The Transformer – Model Architecture

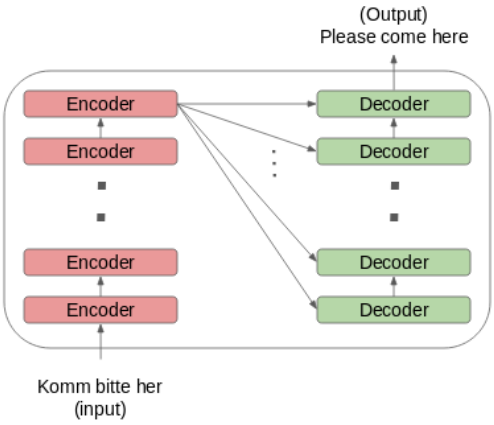
The above image is a superb illustration of Transformer's architecture. Let's first focus on the **Encoder** and **Decoder** parts only.

Now focus on the below image. The Encoder block has 1 layer of a **Multi-Head Attention** followed by another layer of **Feed Forward Neural Network**. The decoder, on the other hand, has an extra **Masked Multi-Head Attention.**



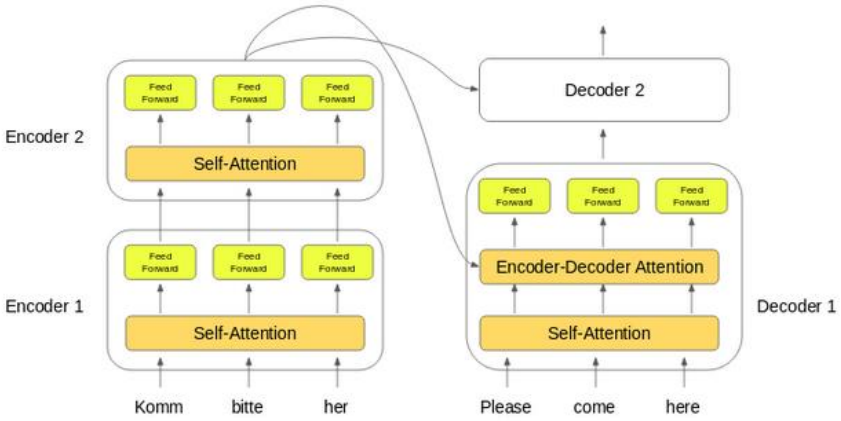
The encoder and decoder blocks are actually multiple identical encoders and decoders stacked on top of each other. Both the encoder stack and the decoder stack have the same number of units.

The number of encoder and decoder units is a hyperparameter. In the paper, 6 encoders and decoders have been used.



Let’s see how this setup of the encoder and the decoder stack works:

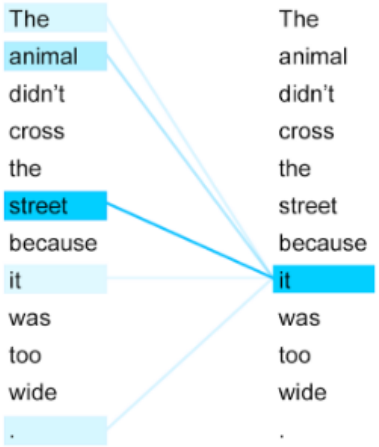
- The word embeddings of the input sequence are passed to the first encoder
- These are then transformed and propagated to the next encoder
- The output from the last encoder in the encoder-stack is passed to all the decoders in the decoder-stack as shown in the figure below:



An important thing to note here – in addition to the **self-attention** and feed-forward layers, the decoders also have one more layer of Encoder-Decoder Attention layer. This helps the decoder focus on the appropriate parts of the input sequence.

You might be thinking – what exactly does this "Self-Attention" layer do in the Transformer? Excellent question! This is arguably the most crucial component in the entire setup so let's understand this concept.

Getting a Hang of Self-Attention



Take a look at the above image. Can you figure out what the term **"it"** in this sentence refers to?

Is it referring to the street or to the animal? It's a simple question for us but not for an algorithm. When the model is processing the word "it", self-attention tries to associate "it" with **"animal"** in the same sentence.

Self-attention allows the model to look at the other words in the input sequence to get a better understanding of a certain word in the sequence. Now, let's see how we can calculate self-attention.

Calculating Self-Attention:

I have divided this section into various steps for ease of understanding.

First, we need to create three vectors from each of the encoder's input vectors:

1. Query Vector
2. Key Vector
3. Value Vector.

These vectors are trained and updated during the training process. We'll know more about their roles once we are done with this section

Next, we will calculate self-attention for every word in the input sequence

Consider this phrase – "Action gets results". To calculate the self-attention for the first word "Action", we will calculate scores for all the words in the phrase with respect to "Action". This score determines the importance of other words when we are encoding a certain word in an input sequence

1. The score for the first word is calculated by taking the dot product of the Query vector (q1) with the keys vectors (k1, k2, k3) of all the words:

Word	q vector	k vector	v vector	score
Action	q_1	k_1	v_1	$q_1 . k_1$
gets		k_2	v_2	$q_1 . k_2$
results		k_3	v_3	$q_1 . k_3$

2. Then, these scores are divided by 8 which is the square root of the dimension of the key vector:

Word	q vector	k vector	v vector	score	score / 8
Action	q_1	k_1	v_1	$q_1 . k_1$	$q_1 . k_1 / 8$
gets		k_2	v_2	$q_1 . k_2$	$q_1 . k_2 / 8$
results		k_3	v_3	$q_1 . k_3$	$q_1 . k_3 / 8$

3. Next, these scores are normalized using the SoftMax activation function:

Word	q vector	k vector	v vector	score	score / 8	Softmax
Action	q_1	k_1	v_1	$q_1 . k_1$	$q_1 . k_1 / 8$	x_{11}
gets		k_2	v_2	$q_1 . k_2$	$q_1 . k_2 / 8$	x_{12}
results		k_3	v_3	$q_1 . k_3$	$q_1 . k_3 / 8$	x_{13}

4. These normalized scores are then multiplied by the value vectors (v1, v2, v3) and sum up the resultant vectors to arrive at the final vector (z1). This is the output of the self-attention layer. It is then passed on to the feed-forward network as input.

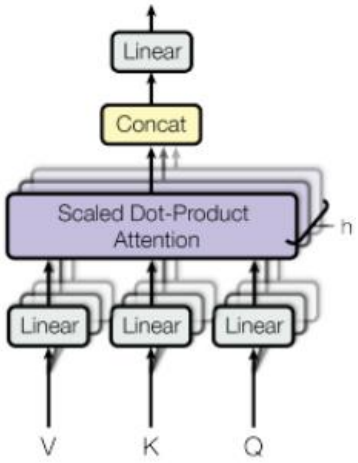
Word	q vector	k vector	v vector	score	score / 8	Softmax	Softmax * v	Sum
Action	q_1	k_1	v_1	$q_1 . k_1$	$q_1 . k_1 / 8$	x_{11}	$x_{11} * v_1$	z_1
gets		k_2	v_2	$q_1 . k_2$	$q_1 . k_2 / 8$	x_{12}	$x_{12} * v_2$	
results		k_3	v_3	$q_1 . k_3$	$q_1 . k_3 / 8$	x_{13}	$x_{13} * v_3$	

So, z1 is the self-attention vector for the first word of the input sequence “Action gets results”. We can get the vectors for the rest of the words in the input sequence in the same fashion:

Word	q vector	k vector	v vector	score	score / 8	Softmax	Softmax * v	Sum#
Action		k_1	v_1	$q_2 . k_1$	$q_2 . k_1 / 8$	x_{21}	$x_{21} * v_1$	
gets	q_2	k_2	v_2	$q_2 . k_2$	$q_2 . k_2 / 8$	x_{22}	$x_{22} * v_2$	z_2
results		k_3	v_3	$q_2 . k_3$	$q_2 . k_3 / 8$	x_{23}	$x_{23} * v_3$	

Word	q vector	k vector	v vector	score	score / 8	Softmax	Softmax * v	Sum#
Action		k_1	v_1	$q_3 . k_1$	$q_3 . k_1 / 8$	x_{31}	$x_{31} * v_1$	
gets		k_2	v_2	$q_3 . k_2$	$q_3 . k_2 / 8$	x_{32}	$x_{32} * v_2$	
results	q_3	k_3	v_3	$q_3 . k_3$	$q_3 . k_3 / 8$	x_{33}	$x_{33} * v_3$	z_3

Self-attention is computed not once but multiple times in the Transformer’s architecture, in parallel and independently. It is therefore referred to as **Multi-head Attention**. The outputs are concatenated and linearly transformed as shown in the figure below:



Multi-Head Attention

Limitations of the Transformer

Transformer is undoubtedly a huge improvement over the RNN based seq2seq models. But it comes with its own share of limitations:

- Attention can only deal with fixed-length text strings. The text has to be split into a certain number of segments or chunks before being fed into the system as input
- This chunking of text causes **context fragmentation**. For example, if a sentence is split from the middle, then a significant amount of context is lost. In other words, the text is split without respecting the sentence or any other semantic boundary

So how do we deal with these pretty major issues? That's the question folks who

Pretrained Language Models:

What is a pretrained model?

A pretrained model is a model that has been trained on a large dataset and can be used as a starting point for other tasks. Pretrained models have already learned the general patterns and features of the data they were trained on, so they can be fine-tuned for other tasks with relatively little additional training data.

In natural language processing (NLP), pre-trained models are often used as the starting point for a wide range of NLP tasks, such as language translation, sentiment analysis, and text summarization. By using a pre-trained model, NLP practitioners can save time and resources, as they don't have to train a model from scratch on a large dataset. Some popular pre-trained models for NLP include BERT, GPT-2, ELMo, and RoBERTa. These models are trained on large datasets of text and can be fine-tuned for specific tasks.

Here are a few excellent pre-trained models for natural language processing (NLP):

1. BERT (Bidirectional Encoder Representations from Transformers):

BERT (Bidirectional Encoder Representations from Transformers) is a state-of-the-art language representation model developed by Google. It is trained on a large dataset of unannotated text and can be fine-tuned for a wide range of natural language processing (NLP) tasks. BERT has achieved state-of-the-art performance on a variety of NLP tasks, such as language translation, sentiment analysis, and text summarization.

BERT is a transformer-based model, which means it uses self-attention mechanisms to process input text. Unlike previous language representation models, BERT is "bidirectional," meaning it considers the context from both the left and the right sides of a token, rather than just the left side as in previous models. This allows BERT to better capture the meaning and context of words in a sentence, leading to improved performance on a variety of NLP tasks. There are also many pre-trained versions of BERT available on the TensorFlow Hub that you can use for your own NLP tasks. These pre-trained models are trained on different datasets and fine-tuned for different tasks, so you can choose the one that is most suitable for your needs.

For example, there is a version of BERT that is fine-tuned for sentiment analysis, and another version that is fine-tuned for question-answering.

2. **GPT-2 (Generative Pretrained Transformer 2):**

   GPT-2 (Generative Pretrained Transformer 2) is a large-scale unsupervised language model developed by OpenAI. It is trained on a massive dataset of unannotated text and can generate human-like text and perform various natural language processing (NLP) tasks. GPT-2 is a transformer-based model, which means it uses self-attention mechanisms to process input text.

   One of the key features of GPT-2 is its ability to generate human-like text. This is useful for applications such as text summarization, language translation, and content generation. GPT-2 can generate text that is coherent and fluent, making it a powerful tool for natural language generation tasks.
   In addition to text generation, GPT-2 can also be fine-tuned for a wide range of NLP tasks, such as sentiment analysis and text classification. It has achieved state-of-the-art performance on a variety of NLP benchmarks, making it a powerful tool for NLP practitioners.

3. **ELMo (Embeddings from Language Models):**

   ELMo (Embeddings from Language Models) is a deep contextualized word representation model developed by researchers at the Allen Institute for Artificial Intelligence. It is trained on a large dataset of unannotated text and can be fine-tuned for a wide range of natural language processing (NLP) tasks.

   One of the key features of ELMo is that it produces word representations that are contextualized, meaning they consider the surrounding words and context in the sentence. This allows ELMo to better capture the meaning and usage of words in a sentence, leading to improved performance on a variety of NLP tasks.

   ELMo can be fine-tuned for a wide range of NLP tasks, including language translation, sentiment analysis, and text classification. It has achieved state-of-the-art performance on several benchmarks, making it a powerful tool for NLP practitioners.

Fine-Tuning and Masked Language Models:

Fine-Tuning in NLP:

1. **Pre-trained Models:** NLP models like BERT (Bidirectional Encoder Representations from Transformers), GPT (Generative Pre-trained Transformer), RoBERTa, etc., are pre-trained on large corpora, learning general language representations. They capture rich contextual information and linguistic nuances.

2. **Task-Specific Adaptation:** Fine-tuning involves taking these pre-trained models and further training them on task-specific data. This adaptation adjusts the model's learned parameters to perform well on specific tasks such as sentiment analysis, question answering, text classification, etc.

3. **Procedure:**
   - **Initialize with Pre-trained Weights:** Start with the parameters learned during pre-training.
   - **Task-Specific Data:** Use a smaller dataset related to the task of interest.
   - **Learning Rate and Layers:** Adjust learning rates, unfreeze and train specific layers or the entire model.
   - **Iterative Training:** Fine-tune the model on the task-specific data, allowing it to learn task-specific patterns.

4. **Benefits:**
   - **Utilizes Pre-trained Knowledge:** Saves time and resources by leveraging knowledge from pre-training.
   - **Better Performance:** Adapts the model to specific tasks, improving its performance on those tasks.

Masked Language Models (MLMs):

1. **Objective:** MLMs are a type of pre-trained language model where the model learns to predict missing or masked words within a sentence.

2. **Training Procedure:**
   - **Masking Tokens:** Randomly mask some of the tokens in the input text.
   - **Prediction Task:** Task the model with predicting the masked tokens based on the context provided by the surrounding words.
   - **Objective Function:** The model is trained to minimize the difference between the predicted and actual masked tokens.

3. **BERT as an Example:**
   - BERT employs a bidirectional Transformer architecture.
   - It masks 15% of the tokens in a sequence.
   - The model aims to predict these masked tokens based on the rest of the input.

4. **Benefits:**
   1. **Captures Contextual Information:** MLMs learn rich contextual representations by understanding relationships between words in a sentence.
   2. **Language Understanding:** Learns semantics, syntax, and linguistic relationships within a sentence.

Relationship:

- **Fine-Tuning with MLMs:** Often, fine-tuning in NLP involves using pre-trained MLMs like BERT or GPT and further training them on task-specific data. This adaptation helps the model to understand the intricacies of the task by adjusting its learned representations without starting from scratch.

In summary, fine-tuning adapts pre-trained models to specific tasks, while Masked Language Models are a type of pre-trained model that learns to predict missing words in a sentence, capturing rich contextual information. Fine-tuning often leverages the capabilities of pre-trained MLMs to improve performance on task-specific data.

Case Study:

1. **Introduction:**
   - **Client Overview:** A multinational e-commerce platform aiming to revolutionize user interactions and expand globally.
   - **Challenge:** Improve user engagement, customer support, sentiment analysis, and enable seamless language translation.

2. **Implementation of ChatGPT:**
   - **Integration of ChatGPT:** Implemented ChatGPT within the platform's customer support system.
   - **Real-time Assistance:** Automated responses for common queries, reducing response time and improving customer satisfaction.
   - **Personalized Interaction:** Tailored responses based on user queries, enhancing the user experience and engagement.

3. **Leveraging GPT Models:**
   - **GPT-Powered Recommendations:** Utilized GPT models for personalized product recommendations based on user preferences.
   - **Enhanced Search Functionality:** Improved search algorithms leveraging GPT's language understanding capabilities for more accurate results.

4. **Sentiment Classification:**
   - **Sentiment Analysis Tool:** Developed an AI-driven sentiment classification system.
   - **Customer Feedback Analysis:** Automated analysis of customer reviews and feedback to gauge sentiment trends.
   - **Customer Insights:** Enabled proactive responses to negative sentiments, enhancing brand reputation.

5. **Language Translation:**
   - **Multilingual Support:** Integrated AI-powered language translation across the platform.
   - **Global Expansion:** Enabled users to access content and communicate in their preferred language.
   - **Seamless Communication:** Facilitated cross-border transactions and interactions with localized content.

6. **Results and Impact:**
   - **Enhanced User Engagement:** Increased user interactions and retention rates by 30%.
   - **Improved Customer Support:** Reduced response time by 50%, leading to higher customer satisfaction.
   - **Better Decision Making:** Insights from sentiment analysis aided in strategic decision-making and product improvements.
   - **Global Reach:** Expanded user base by 40% in non-native English-speaking regions due to language translation support.

7. **Conclusion:**
   - **Future Prospects:** Continuous refinement and updates to AI-powered tools for better accuracy and performance.
   - **Potential Expansion:** Plan to integrate AI for more personalized experiences and predictive analytics.

8. **Key Takeaways:**
   - **AI-Powered Tools:** Significantly improve user experiences, customer support, and engagement.
   - **Sentiment Analysis and Translation**: Crucial for understanding user sentiments and expanding global reach.
   - **Continuous Innovation:** Essential to stay ahead in delivering enhanced AI-driven services.

This case study showcases the transformative impact of AI-powered tools like ChatGPT, GPT models, sentiment classification, and language translation in enhancing user experiences, expanding global outreach, and improving customer interactions for a multinational e-commerce platform.